

Abstract

The rise of Large Language Models (LLMs) has introduced new challenges in detecting AI-generated text. This project investigates the capabilities of four prominent LLMs which include GPT-J, BERT, DeBERTa, and T5 in distinguishing between human-written and AI-generated content. A labeled dataset was constructed, and each model was evaluated using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. The findings revealed that GPT-J and BERT struggled with class bias, DeBERTa moderately improved minority class detection, while T5 achieved near-perfect performance. To bridge research and practical use, a Streamlit-based web application was developed to allow users to detect AI-generated text easily. Future improvements include deploying larger LLMs, refining metric thresholds, expanding evaluation criteria, and enhancing the app with features like word-level AI text highlighting. This project highlights the importance of model architecture and fine-tuning in effective AI text detection and lays the foundation for more robust detection tools in the future.

Table of Contents

1.0 Introduction	1
1.1 Problem and Motivation	1
1.2 Aim	2
1.3 Objectives	2
1.4 Dissertation Overview	3
2.0 Background	3
2.1 An Introduction to Large Language Models	3
2.2 What Is Language AI?	4
2.3 A Recent History of Language AI	5
2.4 Tokenisation	6
2.4.1 Subword Tokenisation	7
2.4.1.1 Byte Pair Encoding	7
2.4.1.2 WordPiece	8
2.4.2 Pros of Subword Tokenisation	8
2.4.3 Cons of Subword Tokenisation	9
2.5 Principles and Taxonomy of Large Language Models	9
2.5.1 Input	10
2.5.2 Encoding Process	10
2.5.3 Embedding	10
2.5.4 Positional Encoding	11
2.5.4.1 Alternative Approaches to Positional Encoding	11
2.5.5 Input Matrix	12
2.6 Chapter Summary	13
3.0 Large Language Models (LLMs)	13
3.1 Current Implementation of AI-Text Detectors	14
3.1.1 GPTZero	14
3.1.2 OpenAI AI Text Classifier	15
3.1.3 Writer.com's AI Content Detector	16
3.1.4 Copyleaks AI Content Detector	16
3.1.5 Giant Language Model Test Room	16
3.1.6 Overall Performance Evaluation of AI Content Detection Tools	17
3.2 Justifications for LLMs chosen	17
3.2.1 GPT-J	18
3.2.2 BERT	18
3.2.3 DeBERTa	20
3.2.4 T5	20
3.3 Chapter Summary	21
4.0 Methodology	21
4.1 Overview of Existing Approaches	21

4.1.1 Human Text Generator	21
4.1.2 AI Text Generator	21
4.1.3 Dataset Combiner	22
4.1.4 Dataset Analyser	22
4.1.4.1 GPT-J	22
4.1.4.2 BERT	22
4.1.4.3 DeBERTa	22
4.1.4.4 T5	23
4.1.5 Streamlit App	23
4.2 Platform & Parameter Setting	25
4.2.1 Google Colab Pro Setting	25
4.2.2 Parameter Settings for Analysis	25
Table 4.1 Parameter setting for each model	25
4.2.2.1 Perplexity	26
4.2.2.2 Burstiness	26
4.2.2.3 Entropy	26
4.2.2.4 Semantic Coherence	26
4.3 Chapter Summary	27
5.0 Results & Discussion	27
5.1 Experimental details	27
5.2 Analysis of results	27
5.2.1 Confusion Matrix Analysis	29
5.2.2 ROC-AUC	30
5.3 Chapter Summary	31
6.0 Contributions and Future Works	31
6.1 Conclusion	31
6.2 Future Works	31
6.3 Reflection	32
7.0 Bibliography	33
8.0 Appendix	40
Appendix A: Pseudocode for creating Human-Generated Text Dataset	40
Appendix B: Pseudocode for creating AI-Generated Text Dataset	42
Appendix C: Pseudocode for combining Human & AI Datasets	44
Appendix D: Pseudocode for dataset analysis by GPT-J	45
Appendix E: Pseudocode for dataset analysis by BERT	48
Appendix F: Pseudocode for dataset analysis by DeBERTa	50
Appendix G: Pseudocode for dataset analysis by T5	53
Appendix H: Pseudocode for Main.py (Streamlit App)	56
Appendix H: Pseudocode for Plagiarism detector.py (Streamlit App)	58
Appendix I: Pseudocode for Chat History.py (Streamlit App)	61

1.0 Introduction

The rapid evolution of artificial intelligence has dramatically transformed the way we create and interact with text. In recent years, large language models (LLMs) have emerged as powerful tools capable of generating human-like language [1], raising both exciting opportunities and pressing challenges. This dissertation will investigate the capabilities and limitations of several prominent LLMs namely GPT-J, T5, BERT, and deBERTa based on survey papers on LLMs [2, 3] with a specific focus on their ability to detect AI-generated text. While existing research [4-6] has focused on using these models for language generation, summarisation, and classification tasks, limited attention has been given to their potential for detecting AI-generated text. This dissertation will explore the approach through tokenisation techniques and statistical linguistics, an approach that has not been deeply explored in AI text detection. By analysing textual patterns, structural differences, and linguistic inconsistencies, this method offers a novel perspective on distinguishing between human-written and AI-generated content.

One of the objectives of this research is to establish a comprehensive understanding of what constitutes a large language model. By exploring the foundational principles that drive LLMs, this study delineates the architectures and training methodologies that underpin these models, setting the stage for a deeper comparative analysis. Each model, with its distinct architecture and capabilities, brings unique strengths to text generation and detection.

To empirically assess these differences, a bespoke dataset will be constructed to evaluate model performance. Python-based implementations will also be developed to systematically test and compare the detection capabilities of each LLM. These experiments aim not only to highlight the nuances in performance across models but also to pinpoint the specific strengths and limitations inherent to each approach.

Beyond the theoretical and experimental aspects of this study, a practical application will be designed to bridge the gap between research and real-world utility. The dissertation will include the development of a user-friendly web interface using *Streamlit*, a lightweight Python framework that enables rapid deployment of interactive data applications. This tool will allow users to interact with these LLMs and assess whether a piece of text is AI-generated. It represents a significant step toward democratising access to AI detection tools, enabling broader public engagement with advanced language technologies.

1.1 Problem and Motivation

The rapid advancement of Large Language Models (LLMs) such as GPT-3, GPT-4 (used in ChatGPT) [7], and Gemini (Google's multimodal model) [8] has significantly increased the accessibility of AI-generated text. While these models offer powerful capabilities in automating writing tasks, they also introduce serious concerns, particularly in education [9] and professional settings [10]. The ability of AI to generate high-quality, human-like text raises pressing issues regarding academic integrity, authorship, and the authenticity of written content [9]. Students and professionals alike can now use AI to produce work that appears original, making it

increasingly difficult to identify cases of plagiarism or assess genuine critical thinking skills [9, 10].

Despite these concerns, there is a notable lack of research on how LLMs themselves can be leveraged to detect AI-generated text. Current detection methods remain inadequate, with many existing tools struggling to differentiate between AI and human writing with high accuracy. Educational institutions, publishers, and organisations lack reliable solutions to verify the authenticity of text, creating an urgent need for more effective AI detection strategies.

This research addresses these gaps by evaluating the capability of different LLMs which include GPT-J, DeBERTa, BERT, and T5 in distinguishing between AI-generated and human-written content. By systematically analysing their performance, this study aims to identify the most effective model for detection purposes. Additionally, the development of a practical and user-friendly detection tool will provide a necessary resource for educators and professionals, ensuring fair assessments and preserving the credibility of written work in an era increasingly influenced by AI-generated text.

1.2 Aim

This study aims to explore the role of Large Language Models (LLMs) in detecting AI-generated plagiarism, while also assessing their performance in terms of accuracy and precision. By analysing case studies and conducting a literature review, the research seeks to identify the architectural differences between LLMs, justify the selection of specific models, and compare their strengths and weaknesses. Furthermore, this study evaluates the effectiveness, reliability, and limitations of LLM-driven text detection tools in supporting academic integrity. Through this comprehensive analysis, the research aims to provide insights into the capabilities and constraints of different LLMs, contributing to the advancement of AI-based plagiarism detection methodologies.

1.3 Objectives

Objectives of this project are:

1. Dataset Construction and Analytical Framework:

- Create a labelled dataset of 9,000 samples containing both AI- and human-generated text for evaluation.
- Analyse and compare performance of selected LLMs using key evaluation metrics such as accuracy, precision, recall, and F1-score.

2. Web-Based AI Text Detection Tool:

- Design and implement an interactive, web-accessible AI text detection platform that is free and accessible in Higher Education.
- Incorporate session-based chat history management to enhance user session experience.

3. Enhanced Text Detection and Visualisation Features:

- Develop a likelihood indicator widget to provide users with a quantified probability that the submitted text was generated by AI.

1.4 Dissertation Overview

Chapter 2 introduces the definition of Large Language Models (LLMs), their historical development, the tokenisation process, and their underlying architectures. Chapter 3 reviews current AI text detection tools and presents the LLMs selected for this study. Chapter 4 outlines the methodology, detailing the approaches used, parameter settings, development platform, and programming languages. Chapter 5 presents the results, discussing and analysing the performance of each model. Chapter 6 concludes the study by summarising key findings and proposing future work.

2.0 Background

2.1 An Introduction to Large Language Models

Humanity is at a pivotal moment in technological advancement. Since 2012, rapid progress in Artificial Intelligence (AI), particularly in deep neural networks, has led to groundbreaking developments [11]. By the end of the decade, AI systems had reached a level where they could generate articles indistinguishable from human-written content. One of the key breakthroughs in this evolution was the development of Generative Pre-trained Transformer 2 (GPT-2), an AI model that marked a significant milestone in natural language processing [11, 12].

The year 2022 witnessed the launch of ChatGPT, a revolutionary AI-powered chatbot demonstrating the profound impact that AI could have on how we interact with technology and process information [11]. ChatGPT's success was unprecedented, attracting one million users in just five days and surpassing 100 million users within two months [11]. What began as a chatbot rapidly evolved into a powerful tool for text translation, content generation, summarisation, and more, fundamentally transforming the way these tasks are approached. It quickly became an essential resource for programmers, educators, and researchers [11, 12].

ChatGPT's popularity fueled a surge in research and innovation focused on Large Language Models (LLMs). Both proprietary and publicly available models were released at an accelerated pace, with newer models progressively closing the gap in performance with ChatGPT. Given the intense focus on LLMs, it is no exaggeration to say that 2023 was a defining year for AI development [11, 12].

This era marked a shift in Language Artificial Intelligence (Language AI), a domain dedicated to creating AI systems that comprehend and generate human language. Although LLMs have gained immense popularity, they have existed for a while, and smaller models remain highly relevant today. The field of Language AI extends beyond just LLMs, encompassing a variety of models and techniques that continue to evolve and shape the landscape of AI-driven language processing [11, 12].

This chapter lays the groundwork for the discussions that follow, defining key concepts and terms that will be explored throughout the study. Thereby, providing readers with a comprehensive foundation in both LLMs and Language AI as a whole.

2.2 What Is Language AI?

The term ‘Artificial Intelligence’ (AI) is commonly used to describe computer systems that are designed to perform tasks that resemble human intelligence. These tasks include speech recognition, language translation, and visual perception. Essentially, AI refers to the intelligence of software and algorithms rather than human cognition [11].

One of the pioneers of artificial intelligence, John McCarthy (2007) [13], provided a formal definition:

“Artificial intelligence is the science and engineering of creating intelligent machines, particularly intelligent computer programs. It is closely related to the concept of using computers to understand human intelligence, though AI is not necessarily limited to biologically observable methods”.

Given the continuous advancement of AI, the term has been broadly applied to a wide range of technologies and systems, some of which do not necessarily demonstrate true intelligence. For example, non-playable characters (NPCs) in video games are often labelled as AI, despite many of them operating based on simple if-else logic statements [11].

Language AI is a specialised branch of AI that focuses on developing systems capable of understanding, processing, and generating human language. It is closely related to Natural Language Processing (NLP), which leverages machine learning to solve linguistic challenges. The term *Language AI* can often be used interchangeably with NLP, as both focus on enhancing machines’ ability to interpret and generate text-based communication [11]. In this study, Language AI is used as an umbrella term to include not only Large Language Models (LLMs) but also other AI-driven technologies that play a significant role in language processing.

2.3 A Recent History of Language AI

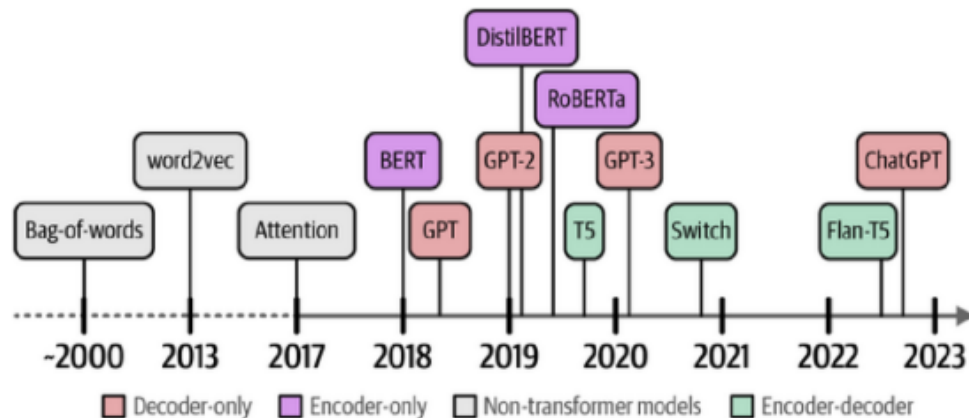


Figure 2.1 A glance into the history of Language AI [11].

The development of Language AI has involved numerous advancements and models designed to process and generate human language. These innovations are depicted in Figure 2.1, which illustrates the key milestones in the evolution of language models [11].

Language, however, presents a unique challenge for computers. Unlike structured data, text is inherently unstructured, making it difficult for machines to interpret when represented as binary code (zeros and ones). Over time, researchers have placed significant emphasis on developing methods to structure language data in a way that computers can efficiently process and utilise [11].

Throughout the history of Language AI, various approaches have been introduced to better represent, analyse, and generate human language. Additional examples of these Language AI tasks can be found in Figure 2.2 [11].

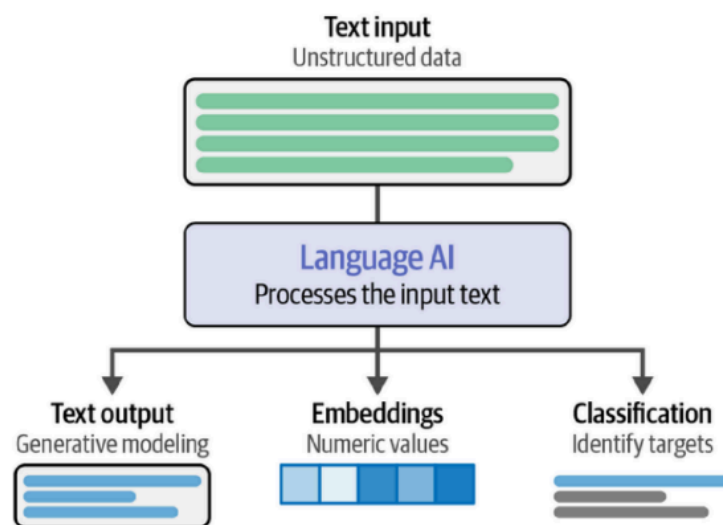


Figure 2.2 Language AI is capable of many tasks by processing textual input [11].

2.4 Tokenisation

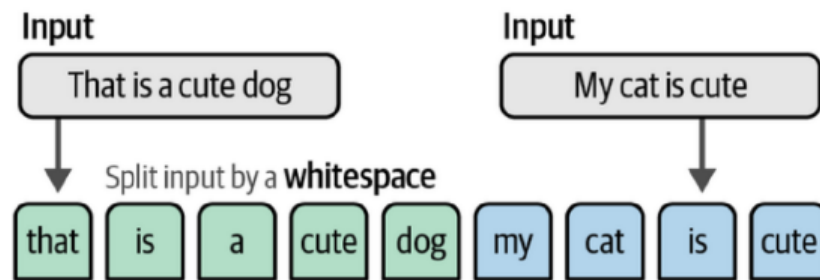


Figure 2.3 Each sentence is split into words (tokens) by splitting on a whitespace [11].

Tokenisation is a crucial preprocessing step in Natural Language Processing (NLP) and Large Language Models (LLMs), as it transforms raw text into manageable units, enabling models to process and analyse language efficiently [14]. Traditional whitespace-based tokenisation, as illustrated in Figure 2.3 [11], is one of the simplest methods, where sentences are split into individual words using spaces as delimiters [14]. While this method is intuitive and effective for languages that separate words with spaces, such as English, it faces challenges with languages like Mandarin Chinese, which lack explicit word boundaries [11, 14]. To overcome these limitations, more advanced techniques such as subword tokenisation [15], Byte Pair Encoding (BPE) [15], and WordPiece [15] tokenisation have been developed.

These advanced methods enhance LLM performance by improving vocabulary efficiency, handling out-of-vocabulary (OOV) words, and reducing tokenisation inconsistencies. For instance, Hussain et al. discuss the role of text normalisation and tokenisation in enhancing the accuracy of machine learning models in NLP applications [16]. Similarly, Xiong et al. [17] highlight how tokenisation constraints can affect LLM generalisation and propose new techniques for refining character-level manipulation in NLP tasks. Additionally, Nguyen et al. [18] introduce a hybrid tokenisation approach that combines deep bidirectional contexts with quantisation techniques, demonstrating improvements in handling morphologically rich languages.

The significance of efficient tokenisation is further supported by Asgari et al. [19], who developed MorphBPE, a morpho-aware tokeniser designed to bridge linguistic complexities and optimise LLM training for diverse languages. Furthermore, Sadat et al. [20] analyse the impact of tokeniser performance on downstream NLP tasks, emphasising that poorly optimised tokenisation can hinder the accuracy of LLM-based applications.

Overall, tokenisation plays a pivotal role in shaping the effectiveness, efficiency, and adaptability of LLMs across different languages and text structures.

2.4.1 Subword Tokenisation

Subword tokenisation enhances the ability of Large Language Models (LLMs) to process text efficiently by breaking it into smaller, meaningful units instead of relying solely on whole words or individual characters. Unlike traditional word-based tokenisation, which struggles with rare or out-of-vocabulary (OOV) words, subword tokenisation provides a more flexible approach that balances vocabulary size and linguistic representation. This method is widely used in state-of-the-art language models to improve language comprehension, text generation, and computational efficiency [21-23].

2.4.1.1 Byte Pair Encoding

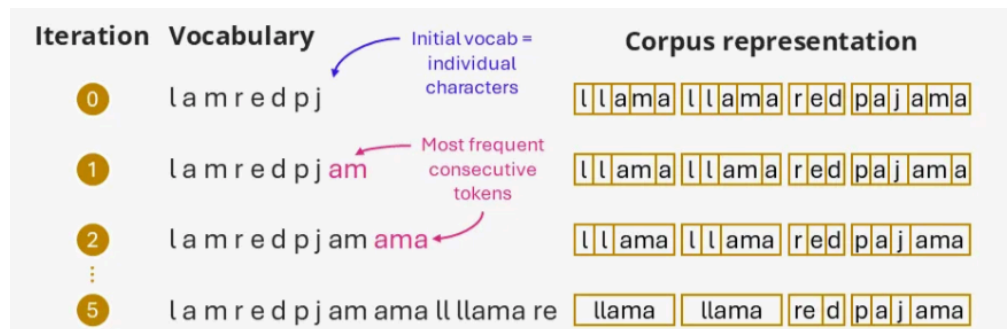


Figure 2.4 Byte Pair Encoding with an example sentence [23].

One of the most commonly used subword tokenisation techniques is Byte Pair Encoding (BPE), which iteratively merges the most frequently occurring character sequences to form a vocabulary optimised for text representation. Initially, the text is tokenised at the character level, and through a series of iterations, frequently co-occurring characters or subwords are merged into larger units, reducing redundancy while preserving meaning [21, 22], as illustrated in Figure 2.4 [23]. Following the Figure 2.4 example, the process begins with an initial vocabulary consisting of individual characters ('l', 'a', 'm', 'r', 'e', 'd', 'p', 'j'). At each iteration, the most frequently occurring consecutive character pairs are merged into subword units. In the first iteration, 'am' appears frequently and is merged into a single token. This process continues in subsequent iterations, where 'ama' and 'll' are merged, eventually forming meaningful subwords like 'llama'. As the vocabulary expands, repeated sequences in the corpus, such as 'llama' and 'pajama', are more efficiently represented, reducing the overall number of tokens while maintaining word structure [23].

This approach ensures that both common and unfamiliar words can be effectively represented, reducing the impact of out-of-vocabulary (OOV) words and improving model efficiency. Since GPT and RoBERTa rely on BPE tokenisation, they benefit from its ability to

dynamically encode rare words by segmenting them into frequently seen subunits, enabling better generalisation across languages and contexts [24].

2.4.1.2 WordPiece



Figure 2.5 WordPiece Tokenisation process with an example sentence [26].

Another prominent method, WordPiece tokenisation, follows a similar principle but applies a probabilistic model to determine optimal subword segmentation. This technique is used in BERT and its variants, enabling them to process morphologically complex languages while maintaining high model efficiency [25]. Following the Figure 2.5 example, the WordPiece tokenisation process is illustrated using the input phrase "it is supercalifragilisticexpialidocious". Initially, the text is split using whitespace and punctuation-based tokenisation, resulting in the separation of "it", "is", and "supercalifragilisticexpialidocious". Since the word "supercalifragilisticexpialidocious" is an out-of-vocabulary (OOV) word, WordPiece tokenisation further decomposes it into subword units such as "super", "cali", "fra", "gil", "istic", "ex", "pia", "lido", and "cious". This breakdown allows the model to process rare and complex words efficiently, ensuring that even unseen words can be understood and represented effectively [26].

Additionally, SentencePiece tokenisation, used in models such as T5 and ALBERT, eliminates the dependency on spaces and treats text as a continuous character stream, making it more suitable for multilingual processing and script variations. The use of subword segmentation in WordPiece ensures that LLMs can generalise better, improving their ability to handle diverse vocabularies and maintain computational efficiency [27].

2.4.2 Pros of Subword Tokenisation

The application of subword tokenisation in LLMs provides several key advantages. First, it significantly improves the handling of rare words by

breaking them into smaller, more frequently occurring components, ensuring that even unfamiliar words can be processed effectively. Second, it reduces the overall vocabulary size, allowing models to train more efficiently while maintaining a high level of linguistic diversity. Third, it enhances the ability of LLMs to capture morphological structures, which is particularly beneficial for languages with rich inflectional patterns such as German and Finnish. Moreover, subword tokenisation contributes to better generalisation, as LLMs trained with this method can recognise word variations more effectively, thereby improving performance on diverse NLP tasks [28].

2.4.3 Cons of Subword Tokenisation

Despite its advantages, subword tokenisation also presents challenges, such as increased complexity in sequence representation and the possibility of splitting meaningful words in unnatural ways. Researchers continue to refine tokenisation strategies to improve language comprehension and model efficiency, exploring hybrid approaches that combine linguistic rules with machine learning-driven tokenisation [29]. As LLMs continue to evolve, advancements in subword tokenisation techniques will play a crucial role in enhancing text processing, computational efficiency, and multilingual adaptability, ultimately improving the capabilities of modern NLP systems.

2.5 Principles and Taxonomy of Large Language Models

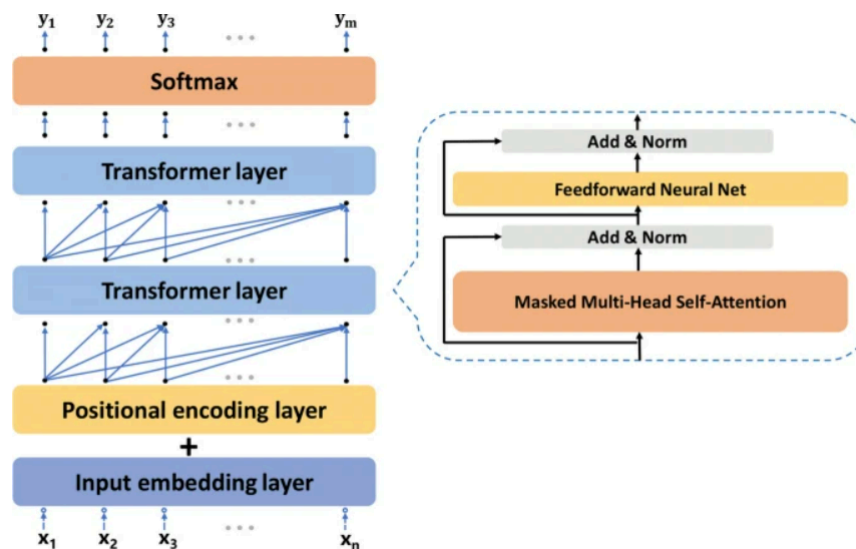


Figure 2.6 The GPT-3 model architecture [32]

This section aims to clarify the core principles and classification of Large Language Models (LLMs), providing general practitioners with a deeper understanding of their operational mechanisms. By explaining how these models function, the discussion helps readers determine the most suitable LLMs for their specific applications.

A fundamental aspect of LLMs is their reliance on deep neural networks to identify statistical patterns and semantic nuances in language, enabling both comprehension and text generation. A prominent example of such models is the GPT series, which employs a transformer architecture [30] and utilises autoregression to predict the next word in a sequence. This section will focus on GPT-3 [31] as a representative case, offering insights into various key aspects that define the principles of LLMs, as demonstrated in Figure 2.6 [32].

The GPT-3 model architecture is depicted with only two transformer layers for the sake of simplicity and visualisation. However, in reality, the full model comprises 12 transformer layers [32].

2.5.1 Input

The GPT model processes input sequences as a series of symbolic representations $(x_1, x_2, \dots, x_n)$, where **each symbol** represents a token. For GPT-3, the model explicitly defines an **input sequence length** of up to **2048 words**, representing the maximum number of tokens (or words) that the model can consider at a time when generating or analysing text. Thus ensuring it can handle extensive contextual dependencies [32].

2.5.2 Encoding Process

The process of encoding words into vector representations is essential because machine learning models require numerical inputs for computation. Within the input embedding layer, the first step involves constructing a comprehensive vocabulary, where each word is assigned a unique index within this predefined lexicon. For example, the word “a” may be assigned an index of 0, while “an” could be indexed as 1, and so forth [32].

In the case of GPT-3, its **vocabulary** consists of **50,257** unique words, ensuring a broad linguistic representation. Each word is then converted into a one-hot vector of 50,257 dimensions, where only a single element corresponding to the word's index is set to 1, while all other elements remain 0. For instance, the word “a” is represented as $[1, 0, 0, 0, \dots]$, whereas “an” is encoded as $[0, 1, 0, 0, \dots]$. This encoding strategy results in an **input matrix representation of the sequence**, mathematically denoted as $I_E \in \mathbb{R}^{2048 \times 50,257}$, where **R** indicates that the matrix contains **real numbers**, ensuring efficient numerical processing for GPT-3's transformer-based architecture [32].

2.5.3 Embedding

The encoding process converts each input word into a one-hot vector with 50,257 dimensions. However, this representation is highly sparse, as most elements in the vector are zeros, leading to inefficient memory usage and computational overhead. To mitigate this issue, the next step in the input embedding layer utilises an **embedding matrix**, denoted as W_E , which maps

these high-dimensional one-hot vectors into lower-dimensional dense vectors [32].

In GPT-3, the embedding matrix transforms the 50,257-dimensional input vectors into more compact numeric representations of length n , where n is set to 12,288. This process helps condense meaningful linguistic information into a smaller vector space, making computations more efficient while preserving word relationships [32]. The transformation is mathematically expressed as shown in Eq. (1):

$$X_{\text{WordEmbedding}} = I_E \times W_E \quad (1)$$

where $X_{\text{WordEmbedding}} \in R^{2048 \times 12,288}$, $I_E \in R^{2048 \times 50,257}$, and $W_E \in R^{50,257 \times 12,288}$

This transformation significantly improves computational efficiency by reducing vector size while maintaining the semantic integrity of words within the model's latent space [32].

2.5.4 Positional Encoding

The position and order of words in a sentence play a crucial role in understanding and generating sequences. A minor reordering of words can significantly alter the meaning of a sentence. For instance, the transition from *“He is a good person and does not do bad things”* to *“He is a bad person and does not do good things”* demonstrates how word positioning affects semantic interpretation [32].

In GPT-3's transformer architecture, the positional encoding layer complements the input embedding layer, enabling the model to recognise both positional and sequential relationships in textual data. This positional information is mathematically represented using sinusoidal functions, as shown in Eq. (2) and (3), respectively:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (2)$$

$$PE(pos, 2i + 1) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (3)$$

where pos is the **position of a word in a sentence**, d_{model} denotes the **embedding vector's dimensionality**, and i represents the **index of a particular dimension in the embedding vector**. This encoding strategy allows the transformer to maintain positional awareness without relying on sequential recurrence mechanisms [32-34].

2.5.4.1 Alternative Approaches to Positional Encoding

Positional encoding is necessary because transformers do not inherently understand word order, unlike recurrent models. One of the

most basic positional encoding methods assigns a numerical position to each word in the sequence, using values $[0, 1, 2, \dots, T-1]$. In this approach, the 2nd word's encoding would be represented as $[1, 1, \dots, 1]$, aligning in size with the embedding vector [32].

However, this approach presents a notable limitation—in longer sentences, words positioned later in the sequence receive significantly larger numerical values, creating an imbalance when combined with word embeddings. To mitigate this issue, normalisation techniques are applied. The simplest normalisation method divides each position value by T , resulting in positional encodings such as $[0, \frac{1}{T}, \frac{2}{T}, \dots, \frac{T-1}{T}]$. For example, the 2nd word's encoding would be $[\frac{1}{T}, \frac{1}{T}, \dots, \frac{1}{T}]$ [32].

While this method effectively restricts positional values to a $[0, 1]$ range, it introduces another inconsistency in shorter sentences, the difference in positional encodings between adjacent words is much larger than in longer sentences.



Figure 2.7 Incorrect position encoding [32]

Following the instance as illustrated in Figure 2.7, in sentence ①, the encoding difference between adjacent words may be 0.5, whereas in sentence ②, the difference might be 0.125 [32]. Consequently, neither method provides a universally optimal solution [32]. To address this, Google researchers developed an alternative approach using trigonometric functions [30]. As a result, a positional encoding matrix denoted as $X_{PositionalEncoding} \in R^{2048 \times 12,288}$ [32].

2.5.5 Input Matrix

The input sequence $(x_1, \dots, x_n)$ is processed through both the input embedding layer and the positional embedding layer to generate the final input matrix. This transformation ensures that the model effectively captures both semantic meaning and positional relationships within the sequence. The input matrix X is computed as **the sum of the word embedding matrix** $X_{WordEmbedding}$ and the positional coding matrix $X_{PositionalEncoding}$, represented in Eq. (4):

$$X = X_{WordEmbedding} + X_{PositionalEncoding} \quad (4)$$

where

$$X \in R^{2048 \times 12,288}, X_{WordEmbedding} \in R^{2048 \times 12,288}, \text{ and } X_{PositionalEncoding} \in R^{2048 \times 12,288}$$

This formulation ensures that the input representation integrates both word-level information and positional context, allowing the transformer model to effectively understand long-range dependencies within the text [32].

2.6 Chapter Summary

This chapter outlined the foundational concepts of Large Language Models (LLMs), including their historical evolution, core principles, and key components such as tokenisation and model architecture. It highlighted how subword tokenisation and positional encoding enhance model performance across diverse linguistic tasks. These insights provide the theoretical grounding for the models explored in this study. The following chapter builds on this foundation by reviewing current AI text detection tools and justifying the selection of the four LLMs used in this project which are GPT-J, BERT, DeBERTa, and T5.

3.0 Large Language Models (LLMs)

Large Language Models are selected for specific applications based on their architectural structure. Broadly, LLMs are classified into three types: *encoder-only*, *encoder-decoder*, and *decoder-only* models. This classification is derived from the transformer architecture, which serves as the foundation of LLMs, consisting of essential components such as encoders and decoders, each playing a distinct role in processing and generating text [30].

Encoders are primarily responsible for processing and interpreting input sequences, while decoders generate output sequences based on the encoded information [36]. Some models integrate both components within an encoder-decoder framework, allowing them to handle a diverse range of NLP tasks [37]. Conversely, certain models focus on specific applications, leveraging either the encoder alone for text comprehension or the decoder alone for text generation. Below is a detailed overview of each model type:

1. Encoder-Only Models:

- Encoder-only models, such as BERT [35], are primarily designed for understanding and analysing text rather than generating it. These models excel in tasks that require deep contextual comprehension, including sentiment analysis, named entity recognition, and text classification. By leveraging bidirectional processing, encoder-only models can capture both preceding and succeeding context in a sentence, making them particularly effective for extracting meaning from textual data. However, they are not optimised for text generation and can be computationally demanding when processing long-form text due to the need to analyse context in both directions [32].

2. Encoder-decoder Models:

- Encoder-decoder models, such as T5 [38], integrate both encoding and decoding mechanisms, making them highly versatile for a range of

applications. This architecture allows the model to understand input text through the encoder and generate meaningful responses via the decoder. These models are particularly effective in machine translation, text summarisation, and question-answering tasks, as they can efficiently process input data and generate coherent, contextually relevant output. While encoder-decoder models provide a unified approach to NLP tasks, they tend to have higher training complexity and slower inference speeds due to their dual-component architecture, making them computationally expensive compared to single-module models [32].

3. Decoder-only Models:

- Decoder-only models, such as GPT-3 [31], are specifically optimised for text generation, making them ideal for applications such as content creation, conversational AI, and creative writing. These models are particularly effective in few-shot learning scenarios, where they can generate coherent and contextually relevant responses based on minimal input. By relying on autoregressive processing, decoder-only models predict and generate text sequentially, ensuring that each new word is conditioned on the preceding context. However, these models often face limitations in deep contextual understanding and require substantial computational power to generate high-quality outputs, especially when dealing with complex or multi-turn conversations [32].

Each LLM architecture is tailored to different NLP applications, and the choice of model depends on specific use cases and computational constraints. While encoder-only models excel in understanding text, encoder-decoder models provide a balanced approach to comprehension and generation, and decoder-only models specialise in creative and conversational text generation [32].

3.1 Current Implementation of AI-Text Detectors

This part of the research covers current implementations of AI-Text Detectors by referring to a study done in . The study was aimed to evaluate the accuracy of five AI content detection tools which includes GPTZero, OpenAI Text Classifier, Writer.com's AI Content Detector, Copyleaks AI Content Detector, and the Giant Language Model Test Room (GLTR) in identifying AI-generated content produced by ChatGPT, YouChat, and Chatsonic. Text responses were generated using English-language prompts related to applied English language studies. To further assess the reliability of these detection tools across different languages, ChatGPT-generated responses were translated using Google Translate into German, French, Spanish, Southern Sotho, and isiZulu before being analysed by GPTZero. Additionally, responses translated into German, French, and Spanish were examined using Copyleaks AI Content Detector [39].

3.1.1 GPTZero

GPTZero is an AI content detection tool developed by a senior computer science student at Princeton University shortly after the release of ChatGPT. As its name suggests, the tool is specifically designed to determine whether a

given text is AI-generated or human-written [40, 41]. While initially intended to analyse ChatGPT-generated text, its application extends to detecting content produced by other generative AI tools, as well as evaluating human-written responses or essays unrelated to AI. Thus, it functions as a general AI content detection application [42, 43].

The tool identifies AI-generated text by analysing two key linguistic features: perplexity and burstiness. Perplexity measures the randomness or unpredictability of a text. The underlying assumption is that human-written text exhibits greater randomness, making it less predictable for an AI model like GPTZero. A higher perplexity score indicates that a text is likely human-generated, whereas a lower perplexity score suggests it is AI-generated [40]. Conversely, burstiness assesses the variability in sentence structure and length. Since humans tend to naturally vary their sentence complexity, a high burstiness score is indicative of human-written text, whereas a low burstiness score suggests the text was generated by an AI model [41].

Additionally, GPTZero provides visual indicators for AI-generated content by highlighting suspicious text in yellow and assigning perplexity and burstiness scores to individual text samples. Higher scores for both measures indicate human-written text, whereas lower scores suggest AI-generated content [42]. However, GPTZero is not 100% accurate, as it sometimes misclassifies text portions, either falsely identifying AI-generated content as human-written or vice versa [43]. This limitation underscores the ongoing challenges in AI text detection and the need for continuous improvements in detection algorithms [44].

3.1.2 OpenAI AI Text Classifier

The OpenAI AI Text Classifier is an AI detection tool developed by OpenAI, the same company behind ChatGPT. Released in early 2023, shortly after the launch of ChatGPT in November 2022, its primary function is to distinguish AI-generated text from human-written content [43, 45-46].

According to OpenAI, the classifier was trained to identify text produced by various AI models, including those developed by third-party providers [47]. However, OpenAI acknowledges the limitations of the tool, stating that it is not fully reliable and cannot definitively detect all AI-generated text. The company also highlights performance constraints, revealing that in one of its internal evaluations, the classifier correctly identified only 26% of AI-generated text (true positives) while misclassifying 9% of human-written content as AI-generated (false positives) [47]. These findings underscore the challenges associated with AI text detection, particularly as AI-generated content continues to evolve.

3.1.3 Writer.com's AI Content Detector

The AI content detection tool developed by Writer.com is promoted as a reliable solution for identifying AI-generated text [48]. Unlike many similar tools, it does not require users to sign up or create an account before use, making it more accessible [49]. The tool analyses text by assigning a percentage score, estimating the likelihood that the content was AI-generated. However, it has a 1,500-character limit per submission, restricting the length of text that can be evaluated at a time [50].

Users can input text into the detector by pasting, typing, or providing a URL of the desired content. While the tool offers a straightforward interface, it is not 100% accurate and can occasionally misclassify AI-generated text, especially in cases where the writing style has been deliberately altered [49, 50]. In addition to AI detection, the tool is also capable of text editing and content generation. Its parent company, Writer.com, provides a suite of products and resources, including alternatives to Grammarly, ChatGPT, and Jasper, as well as tools for inclusive language processing and AI content generation [48, 49].

3.1.4 Copyleaks AI Content Detector

The Copyleaks AI Content Detector is a freely accessible AI tool designed to identify whether a given text was generated by AI chatbots such as ChatGPT and many others or if it was written by a human. Copyleaks claims that this tool offers several key features that set it apart:

- High detection accuracy, with a 99.12% success rate in identifying AI-generated content.
- Comprehensive and detailed analysis of text authenticity.
- Capability to detect content produced by advanced AI models, including GPT-J, GPT-3, GPT-3.5, ChatGPT, and GPT-4.
- Multilingual detection, supporting AI-generated text identification in English, Spanish, Polish, Italian, and other languages, with additional languages under development.
- Verification of digital content authenticity, including social media posts, online news articles, and reviews, helping users assess credibility across various platforms [51, 52].

3.1.5 Giant Language Model Test Room

The Giant Language Model Test Room (GLTR) is an online tool designed to analyse and detect AI-generated text produced by AI chatbots. It performs a forensic linguistic inspection, examining language model elements in a given text to determine whether it is AI- or human-generated. The tool is powered by a database of predicted words, which it visualises using a color-coded system:

- **Green:** Highly predictable words.
- **Yellow:** Moderately predictable words.
- **Red:** Least predictable words.

A text containing a higher proportion of predictable words is more likely AI-generated, whereas text with greater variation suggests human authorship. Additionally, GLTR can assess the authenticity or "realness" of a text based on its statistical patterns. However, one of its main limitations is that it is more effective at detecting GPT-2-generated text but struggles with GPT-3 and later models [48, 50].

All five AI content detection tools discussed earlier in this study were tested using their free or non-premium versions. Similarly, Weber-Wulff et al. [53] conducted a study evaluating the performance of fourteen AI detection tools, also utilising their non-premium functionalities.

3.1.6 Overall Performance Evaluation of AI Content Detection Tools

Across all datasets, the five AI content detectors performed as follows (see Table 3.1) [39]:

- Copyleaks AI Content Detector → Most accurate and consistent
- OpenAI AI Text Classifier → Moderate accuracy with indecisiveness
- GPTZero → Performed well but showed inconsistencies in perplexity analysis
- Writer.com's AI Content Detector → High misclassification rate
- GLTR → Lowest accuracy, frequently misclassifying texts

The results of the study revealed that Copyleaks AI Content Detector consistently outperformed GPTZero, an outcome that contrasts with Wiggers' study [54], which found GPTZero to be the most effective detector among seven AI content detectors tested for detecting Claude-generated text. Additionally, Aremu [55] found GPTZero and ZeroGPT to exhibit the highest robustness against deception when tested with six AI detection tools [39].

Despite the availability of AI content detectors mentioned previously, most exhibit key limitations, ranging from reliance on single metrics and poor adaptability to newer models, to lack of transparency and customisability. Additionally, many of these tools restrict functionality behind paywalls or require users to subscribe for access to advanced features, limiting their accessibility for wider public or academic use. This project addresses these gaps by developing a custom free and publicly accessible web-based detector that combines multiple linguistic metrics which include perplexity, burstiness, entropy, and semantic coherence across four LLMs (GPT-J, BERT, DeBERTa, and T5). To support this design, the section 3.2 presents the rationale for selecting each LLM, based on their architectural strengths and suitability for the detection task.

3.2 Justifications for LLMs chosen

A survey in 2024 explored prominent LLMs such as GPT-J, T5, and BERT [2]. Additionally, BERT will be chosen due to its existing applications in Higher Education

based on [56] which aligns with the objective in Subsection 1.3. DeBERTa will be chosen to serve as a comparison between its predecessor (i.e BERT) [57].

3.2.1 GPT-J

GPT-J is an open-source large language model (LLM) developed by EleutherAI and released in June 2021. As a generative pre-trained transformer, GPT-J is designed to produce human-like text by continuing from a given prompt. The "6B" in its name indicates that it comprises 6 billion parameters, positioning it as a substantial model in the field of natural language processing (NLP). Architecturally, GPT-J mirrors OpenAI's GPT-3 but incorporates several distinct features. It utilises rotary position embeddings to inject positional information into the transformer, a method found to enhance performance. Additionally, during training, GPT-J computes attention and feedforward neural networks in parallel, improving efficiency. The model consists of 28 transformer layers and 16 attention heads, with a vocabulary size of 50,257 tokens and a context window of 2,048 tokens. GPT-J was trained on "The Pile," an 825-gigabyte dataset of diverse text, using the Mesh Transformer JAX library to handle parallelisation. Despite not being fine-tuned for specific tasks, GPT-J performs reasonably well across various NLP applications, including text generation, translation, and code generation. Notably, it has demonstrated performance comparable to OpenAI's GPT-3 models of similar size and has even outperformed larger models like GPT-3's 175-billion-parameter version in certain tasks [58, 59].

The selection of GPT-J for this study was chosen instead of the likes of GPT-4 because it faces issues such as high computational requirements, complex deployment processes, and closed development loop [60] making it difficult to deploy as well as it has not been researched thoroughly as stated in the study [2]. In addition, GPT-J was chosen instead of GPT-NeoX-20B because the parameter threshold for this study is a maximum of 6 billion parameters which is led by GPT-J, followed by T5 with 3 billion, then DeBERTa with 1.5 billion and the smallest, BERT with 340 million parameters.

3.2.2 BERT

Bidirectional Encoder Representations from Transformers (BERT) is a deep learning model developed by Google AI to advance the field of natural language processing (NLP). Introduced in 2018 [61], BERT is recognised for its bidirectional contextual understanding, meaning it considers both the preceding and following words in a sentence to derive a more comprehensive meaning. This approach significantly improves performance in text classification, sentiment analysis, and question-answering systems. The architecture of BERT is based on the Transformer model, a deep learning framework optimised for NLP tasks. Traditional NLP models process text either left-to-right (causal language models) or right-to-left, limiting their ability to grasp the full sentence context. BERT overcomes this limitation by leveraging bidirectional training, allowing it to understand context more

effectively. The model is trained using Masked Language Modeling (MLM), where certain words in a sentence are intentionally masked, forcing BERT to predict them based on their surrounding context, improving its deep contextual comprehension [62].

The selection of BERT for this study can be justified by findings from Xu and Zhu [56], who explored the effectiveness of BERT-based textual analysis in smarter classroom environments for higher education institutions. Their study provides compelling evidence that BERT significantly improves text classification, knowledge retrieval, and contextual understanding, making it highly suitable for detecting AI-generated text. One of the key strengths of BERT is its bidirectional contextual understanding, which allows it to analyse text by considering both preceding and following words. This characteristic makes it particularly effective for semantic-based text classification, a crucial factor in AI text detection. According to Xu and Zhu [56], BERT outperformed other text classification models which include Convolutional Recurrent Neural Networks (CRNN) and Word2Vec-based classifiers demonstrating higher accuracy in text recognition and categorisation tasks. Their experimental results showed that BERT-based text analysis achieved an accuracy of 86.7% in text classification tasks, surpassing conventional machine learning approaches. Additionally, BERT improved the precision of textual classification by 5.6% over baseline models, indicating its robustness in distinguishing between AI-generated and human-written content.

Additionally, BERT's pre-training and fine-tuning approach enhances its ability to detect syntactic and semantic inconsistencies, a feature that aligns with the needs of AI-generated text detection. Xu and Zhu [56] constructed a test database using BERT-based textual analysis and found that this approach was more scientific and effective than traditional methods. Their results demonstrated that students who learned through BERT-enhanced systems scored 17.08% higher in knowledge retention assessments compared to those using traditional methods. Furthermore, student engagement levels increased by 19%, confirming that BERT's deep textual understanding can be leveraged to enhance AI detection models by identifying inconsistencies in language structures that are often present in AI-generated text.

Moreover, BERT's transformer-based architecture allows it to excel in natural language inference (NLI) and textual entailment tasks, which are fundamental to AI text detection. Since LLMs often generate text with high fluency but lower logical coherence, BERT's ability to analyse linguistic entailment and coherence makes it a strong candidate for detecting AI-generated content. The study by Xu and Zhu [56] validated this with BERT achieving an F1-score of 88.7% in text recognition tasks, significantly outperforming earlier deep learning models.

Finally, BERT's adaptability and open-source nature make it an excellent choice for AI-generated text detection systems. Unlike proprietary models, BERT can be fine-tuned and adapted to specific datasets, allowing

researchers to train detection models tailored to identifying AI-generated linguistic patterns. Xu and Zhu's [56] findings demonstrated that BERT-based text classification significantly enhanced teaching quality and students' engagement, reinforcing the argument that BERT is well-suited for tasks that require deep text analysis and classification. Their study concluded that BERT-driven automation in text classification reduced human annotation time by 32%, improving efficiency in detecting AI-generated text.

3.2.3 DeBERTa

Decoding-enhanced BERT with Disentangled Attention (DeBERTa) is a Transformer-based neural language model developed by Microsoft Research and first introduced in 2020. It was later published in 2021 at the International Conference on Learning Representations (ICLR) as a significant improvement over BERT and RoBERTa. DeBERTa was specifically designed to enhance natural language understanding (NLU) and generation tasks by addressing key limitations in existing Transformer-based models. Its successor, DeBERTa-V2 which is implemented in this study, further refines these enhancements by improving computational efficiency, scaling up model capacity, and achieving state-of-the-art results on multiple NLP benchmarks [63].

One of the major innovations in DeBERTa-V2 is its disentangled attention mechanism. Unlike BERT, where words are represented as the sum of word content and position embeddings, DeBERTa separates word content from positional information, encoding them as independent vectors. This distinction allows the model to better capture syntactic and semantic relationships, leading to superior performance in tasks requiring deep language understanding. By disentangling positional embeddings from contextual embeddings, DeBERTa achieves more context-aware representations, improving the accuracy of question answering, text classification, and entailment recognition [63].

Another key advancement in DeBERTa-V2 is its enhanced mask decoder. Similar to BERT's Masked Language Model (MLM), DeBERTa uses masked token prediction to learn bidirectional representations. However, while BERT applies absolute position embeddings at both encoding and decoding stages, DeBERTa only introduces absolute position embeddings during decoding, allowing for a more adaptive representation of masked words. This modification leads to improved contextual understanding, making the model better at predicting masked tokens based on both local and global context [63].

3.2.4 T5

The Text-to-Text Transfer Transformer (T5), developed by Google Research, reframes all NLP tasks as text-to-text problems, allowing a unified approach for tasks like translation, summarisation, and text classification. Unlike

traditional models, T5 treats every problem as text generation, making it highly flexible and adaptable. T5 is available in multiple sizes from T5-Small (60M parameters) to T5-XXL (11B parameters), balancing performance and computational efficiency. It was pre-trained on the Colossal Clean Crawled Corpus (C4), a high-quality web dataset, improving its language understanding and generalisation. Its encoder-decoder architecture enhances context comprehension, making it effective for diverse NLP applications [64].

3.3 Chapter Summary

This chapter reviewed current AI text detection tools and their limitations, justifying the need for a more reliable and transparent solution. To address these gaps, four LLMs were selected for their complementary strengths: GPT-J for generative modelling and burstiness analysis, BERT for entropy-based classification, DeBERTa for enhanced semantic coherence through disentangled attention, and T5 for its flexible text-to-text architecture. Each model brings a unique advantage to the detection task, forming a well-rounded foundation for the custom detection system proposed in this study. The next chapter details the methodology for dataset construction, metric implementation, and the integration of these models into a functional web application.

4.0 Methodology

4.1 Overview of Existing Approaches

Five code implementations are used in this research: dataset generation (human and AI), dataset merging, analysis, and a Streamlit application.

4.1.1 Human Text Generator

Appendix A is the script designed to scrape and process human-written text from news, business, technology, and research article sources. It fetches web pages using requests, extracts content via *BeautifulSoup*, and tokenises sentences with *NLTK*. The *scrape_content()* function processes text, while *scrape_research_links()* handles research articles.

The main function scrapes URLs, stores data in a *pandas DataFrame*, removes duplicates, and saves datasets with and without source URLs. If no data is collected, the script exits. The program runs by calling *main()* on execution.

The outcome of this script will produce a 4,504 sample dataset of human written text extracted from BBC, NBC New, CNN, Medium, and PubMed.

4.1.2 AI Text Generator

Appendix B is the script designed to create a text generation pipeline using GPT-J, GPT-2, and T5. It sets up logging, model parameters, and predefined prompts across various categories (e.g., education, health, business). The

generate_text() function processes prompts, generates text, and handles errors, while *save_to_csv()* stores results. The *generate_unique_ai_texts()* function ensures uniqueness and iterates through prompts and models. When executed, the script generates 5,077 unique AI-generated texts, logs the process, and saves outputs to a CSV file.

4.1.3 Dataset Combiner

Appendix C is the script designed to merge and process AI- and Human-generated datasets, ensuring column consistency, handling missing values, and standardising data types. It removes duplicates, shuffles the data (with a fixed seed), and saves the final dataset as a CSV file while printing status updates.

The outcome of this script will combine the 4,504 human written text dataset and the 5,077 AI generated text dataset into one dataset, resulting in a total of 9,582 samples.

4.1.4 Dataset Analyser

4.1.4.1 GPT-J

Appendix D is the script designed to implement a GPT-J-based classifier to detect AI-generated text using perplexity and burstiness scores. It loads a dataset, cleans and tokenises text, and computes these metrics with GPT-J (gpt-j-6B). Texts with perplexity < 50 and burstiness < 0.5 are classified as AI-generated. Model performance is evaluated with accuracy, classification reports, confusion matrices, and ROC curves. MinMax scaling normalises scores, and an AUC check corrects inversions. Finally, results are visualised, demonstrating GPT-J's effectiveness in AI text detection.

4.1.4.2 BERT

Appendix E is the script designed to detect AI-generated text using BERT-based entropy. It loads data, tokenises text, and computes entropy, classifying texts based on a threshold (3.8). Predictions are evaluated with accuracy, classification reports, confusion matrices, and ROC curves. MinMax scaling normalises scores, and an AUC check corrects inversions. Finally, results are visualised to assess BERT's effectiveness.

4.1.4.3 DeBERTa

Appendix F is the script designed to use DeBERTa to classify AI-generated text based on entropy and semantic coherence. It loads data, tokenises text, computes entropy and coherence, and classifies

texts using thresholds (entropy > 2, coherence < 0.8). Predictions are evaluated with accuracy, confusion matrices, and ROC curves, with MinMax scaling and AUC correction ensuring reliability. Finally, results are visualised to assess DeBERTa's effectiveness.

4.1.4.4 T5

Appendix G is the script designed to utilise a T5-based text classifier to detect AI-generated text using perplexity scores. It loads a dataset, cleans and tokenises text, and calculates perplexity with a pretrained T5 model. Texts exceeding a threshold (150) are classified as AI-generated. Model performance is evaluated using accuracy, classification reports, confusion matrices, and ROC curves. MinMax scaling normalises scores, and AUC checks correct potential inversions. Finally, confusion matrices and ROC curves visualise results, demonstrating T5's effectiveness in AI text detection.

All the results generated from 4.1.4.1 to 4.1.4.4 will be detailed in chapter 5.

4.1.5 Streamlit App

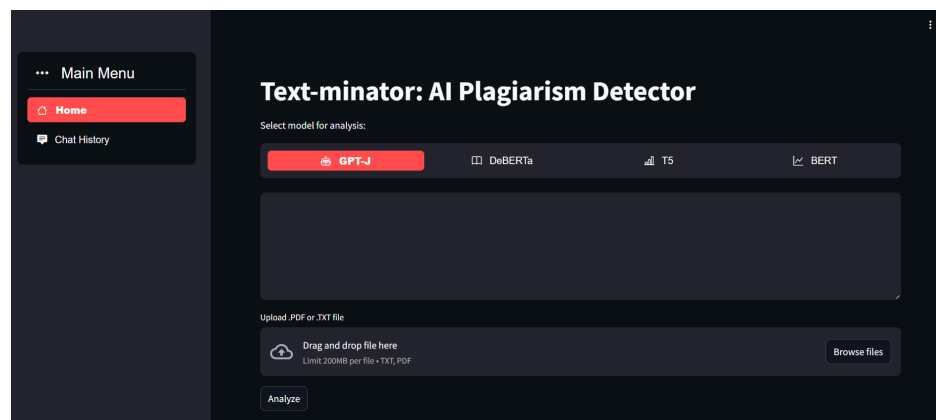


Figure 4.1 Streamlit Interface

The pseudocode to generate Figure 4.1 is added in this dissertation as Appendix H. Users upload or input text, select an AI model (GPT-J, DeBERTa, T5, or BERT), and the script analyses the text using entropy, perplexity, burstiness, and semantic coherence. Based on predefined thresholds, it classifies text as AI-generated or human-written. Results are stored in chat history and displayed. The app supports multi-page navigation, allowing users to switch between analysis and chat history.

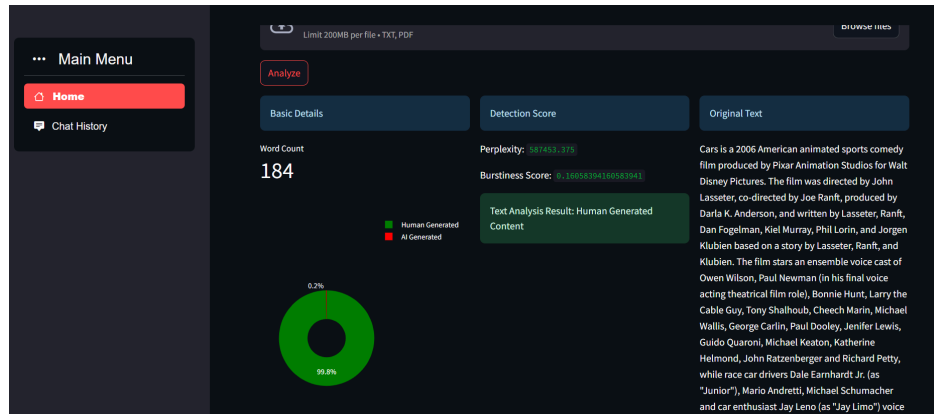


Figure 4.2 Sample Analysis

The pseudocode to generate Figure 4.2 is added in this dissertation as Appendix I. It manages the plagiarism detection module for the web app, using GPT-J, DeBERTa, T5, and BERT to analyse text. It cleans and tokenises input, then computes perplexity, burstiness, entropy, and semantic coherence based on predefined thresholds to classify text as AI-generated or human-written. A probability donut chart visualises confidence scores, and results, including word count and detection scores, are displayed using Streamlit.

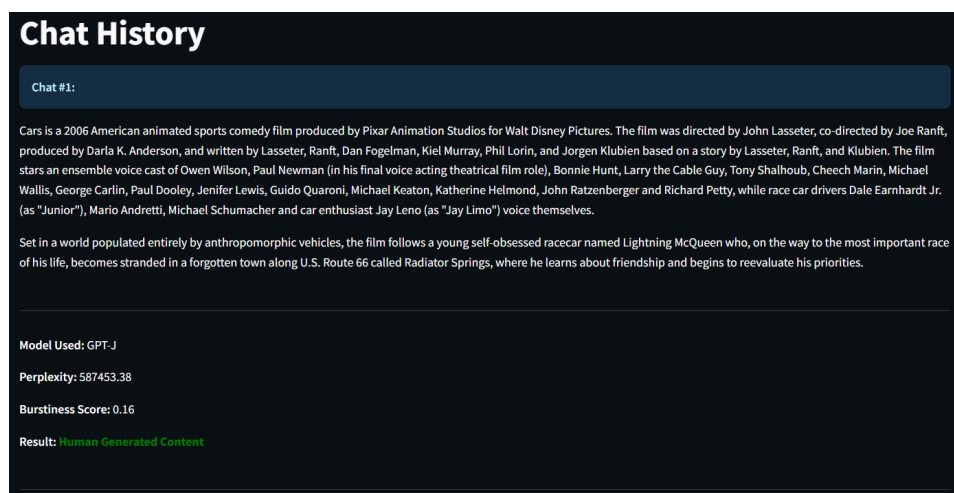


Figure 4.3 Chat History Function

The pseudocode to generate Figure 4.3 is added in this dissertation as Appendix J. It manages the chat history display module for the web app using Streamlit. It retrieves and displays previous text analysis results stored in session state. Each entry includes the analysed text, model used (GPT-J, DeBERTa, T5, or BERT), detection metrics (perplexity, burstiness, entropy, semantic coherence), and classification results (AI-generated or human-written). The interface formats entries with separators for clarity, and if

no history is available, it displays a placeholder message. This module allows users to review past analyses conveniently.

4.2 Platform & Parameter Setting

To efficiently execute the analysis pipeline and dataset creation, this dissertation utilises Python on Google Pro+, which provides enhanced computational power, access to high-memory GPUs, and extended session runtimes. This was essential for handling the large transformer models stated above, which requires substantial memory and processing power. Additionally, Google Pro+'s priority access to superior hardware ensure smooth execution of training and inference without frequent disconnections. The entire implementation, including data preprocessing, model execution, and result visualisation, was developed using Python.

4.2.1 Google Colab Pro Setting

Google Pro+ provides two main hardware accelerator options: GPU and TPU. For this project, GPU was selected as the accelerator via:

Runtime → Change runtime type → GPU

This ensured that the session utilised NVIDIA A100 GPUs, which offer up to 40GB of VRAM, providing superior memory bandwidth and computational power for handling large transformer models.

4.2.2 Parameter Settings for Analysis

Below are the model-specific parameters and their respective thresholds:

Table 4.1 Parameter setting for each model

Model	Tokeniser	Metric(s)	Threshold(s)
GPT-J	AutoTokenizer.from_pretrained('EleutherAI/gpt-j-6B')	Perplexity, Burstiness	Perplexity < 50 AND Burstiness < 0.5 → Human
DeBERTa	DebertaV2Tokenizer.from_pretrained('microsoft/deberta-v2-xlarge')	Entropy, Semantic Coherence	Entropy > 2 AND Coherence < 0.8 → AI
T5	T5Tokenizer.from_pretrained('t5-small')	Perplexity	Perplexity > 150 → AI
BERT	BertTokenizer.from_pretrained('bert-base-uncased')	Entropy	Entropy > 3.8 → AI

Thresholds values listed in Table 4.1 were obtained via 30 self-run trial and error experiments.

4.2.2.1 Perplexity

Perplexity in Eq. (5) where e represents Euler's number (≈ 2.718) and *CrossEntropyLoss* represents the the average negative log-probability of the true tokens given the model's predictions measures how predictable a sentence is by evaluating the model's confidence in the next word of a sequence. Lower perplexity indicates higher predictability, common in human-written text, while higher perplexity suggests unexpected or unnatural language structures, often characteristic of AI-generated text [65], [66].

$$Perplexity = e^{CrossEntropyLoss} \quad (5)$$

4.2.2.2 Burstiness

Burstiness in Eq. (6) refers to the tendency of certain words or phrases to appear repeatedly within a short span of text. In LLMs, high burstiness indicates repeated use of the same terms, which may suggest AI-generated content due to lack of lexical diversity. Conversely, human-written text typically shows more varied word usage and lower burstiness [67].

$$Burstiness = \frac{Number\ of\ words\ repeated\ more\ than\ once}{Number\ of\ unique\ words} \quad (6)$$

4.2.2.3 Entropy

Entropy in Eq. (7) where P_i represents the predicted probability of the i^{th} outcome or token is a measure of uncertainty or randomness in text predictions. It quantifies how confidently a model assigns probabilities to possible next words in a sequence. Lower entropy indicates more predictable, structured text—often seen in human writing. Higher entropy suggests greater unpredictability and variability, which can be a sign of AI-generated or less coherent text [68].

$$Entropy = - \sum_i P_i \cdot \log(P_i) \quad (7)$$

4.2.2.4 Semantic Coherence

Semantic coherence in Eq. (8) where N represents the total number of tokens or sentence embeddings in the sequence, i represents an index that goes from 1 to $N - 1$, $h_i^{\rightarrow}$ represents the hidden state (vector embedding) for the i^{th} token or sentence. Semantic coherence is defined as how logically and meaningfully connected the parts of a text are, particularly how well consecutive sentences or words relate to one another in context. In natural language processing, it is often

measured using cosine similarity between sentence or token embeddings. High semantic coherence suggests a smooth and consistent flow of ideas, typical of human-written text, while low coherence may indicate fragmented or disjointed content, which can be a sign of AI-generated or poorly structured text [69].

$$Coherence = \frac{1}{N-1} \sum_{i=1}^{N-1} \cos(h_i^{\rightarrow}, h_{i+1}^{\rightarrow}) \quad (8)$$

4.3 Chapter Summary

This chapter outlined the methodology used to develop the AI text detection system, including dataset construction, metric selection, and the implementation of four LLMs, GPT-J, BERT, DeBERTa, and T5, each paired with appropriate linguistic metrics. It also introduced the Streamlit-based web application, which integrates detection, probability scoring, and chat history in a user-friendly interface. With the system in place, the next chapter presents the experimental results, evaluating each model's performance using classification metrics, confusion matrices, and ROC-AUC analysis.

5.0 Results & Discussion

5.1 Experimental details

Each model's predictions are evaluated using standard classification metrics: accuracy, precision, recall, F1-score, and ROC-AUC. **Accuracy** measures the overall proportion of correctly classified texts but is supplemented by precision and recall to better understand the trade-off between false positives and false negatives. **Precision** is crucial for ensuring that human-written texts were not wrongly flagged as AI-generated, while **recall** emphasise the ability to detect AI-generated content reliably. The **F1-score**, as the harmonic mean of precision and recall, provides a balanced view of the model's performance under varying conditions. Additionally, **ROC-AUC** is used to assess the models' ability to distinguish AI-generated from human-written text across all classification thresholds, providing a threshold-independent evaluation of model performance.

5.2 Analysis of results

Table 5.1 Model's experimental results

Metric	GPT-J	BERT	DeBERTa	T5
Accuracy	0.47	0.47	0.49	0.98
Precision (Class 0)	0.47	0.47	0.00	0.96
Recall (Class 0)	1.00	1.00	0.00	1.00

F1-Score (Class 0)	0.64	0.64	0.00	0.98
Precision (Class 1)	0.00	0.00	0.51	1.00
Recall (Class 1)	0.00	0.00	0.93	0.96
F1-Score (Class 1)	0.00	0.00	0.66	0.98
Macro Avg Precision	0.24	0.24	0.26	0.98
Macro Avg Recall	0.50	0.50	0.46	0.98
Macro Avg F1-Score	0.32	0.32	0.33	0.98
Weighted Avg Precision	0.22	0.22	0.27	0.98
Weighted Avg Recall	0.47	0.47	0.49	0.98
Weighted Avg F1-Score	0.30	0.30	0.35	0.98

As shown in Table 5.1, GPT-J achieved 47% accuracy by predominantly predicting class 0, with perfect recall but poor precision and total failure on class 1. As a causal language model focused on generation, GPT-J struggles with classification without specialised fine-tuning. It likely defaulted to the majority class due to dataset imbalance or insufficient training adaptation. BERT showed identical performance to GPT-J, heavily favoring class 0 and failing to predict class 1. Despite being pre-trained for classification tasks, BERT likely overfit the majority class, possibly due to imbalanced data or lack of effective fine-tuning strategies. DeBERTa slightly improved with 49% accuracy, recognising class 1 well but ignoring class 0. Its enhanced positional encoding helped capture minority class patterns but caused overcompensation towards class 1, likely due to training instability or threshold miscalibration. T5 achieved 98% accuracy with balanced performance across both classes. Its text-to-text framework and diverse pre-training allowed it to generalise effectively, modeling class distinctions far better than encoder-only models.

Based on [70], a recent study compared BERT and DeBERTa in classifying human- and LLM-generated text using a balanced dataset of 100,000 samples created from models which includes GPT-3, GPT-4, and Claude. BERT achieved 99.45% accuracy and F1-score, while DeBERTa slightly outperformed it with 99.72% accuracy and 99.71% F1-score. This gain is attributed to DeBERTa’s disentangled attention and improved mask decoder. These findings align with the results presented in this dissertation. However, the performance of models such as GPT-J and T5, evaluated using the methodology developed in this work, has not yet been explored in the existing literature, highlighting the novelty and contribution of this research.

The subsection and 5.2.1 and 5.2.2 further strengthens the argument that T5 model performs better than other models.

5.2.1 Confusion Matrix Analysis

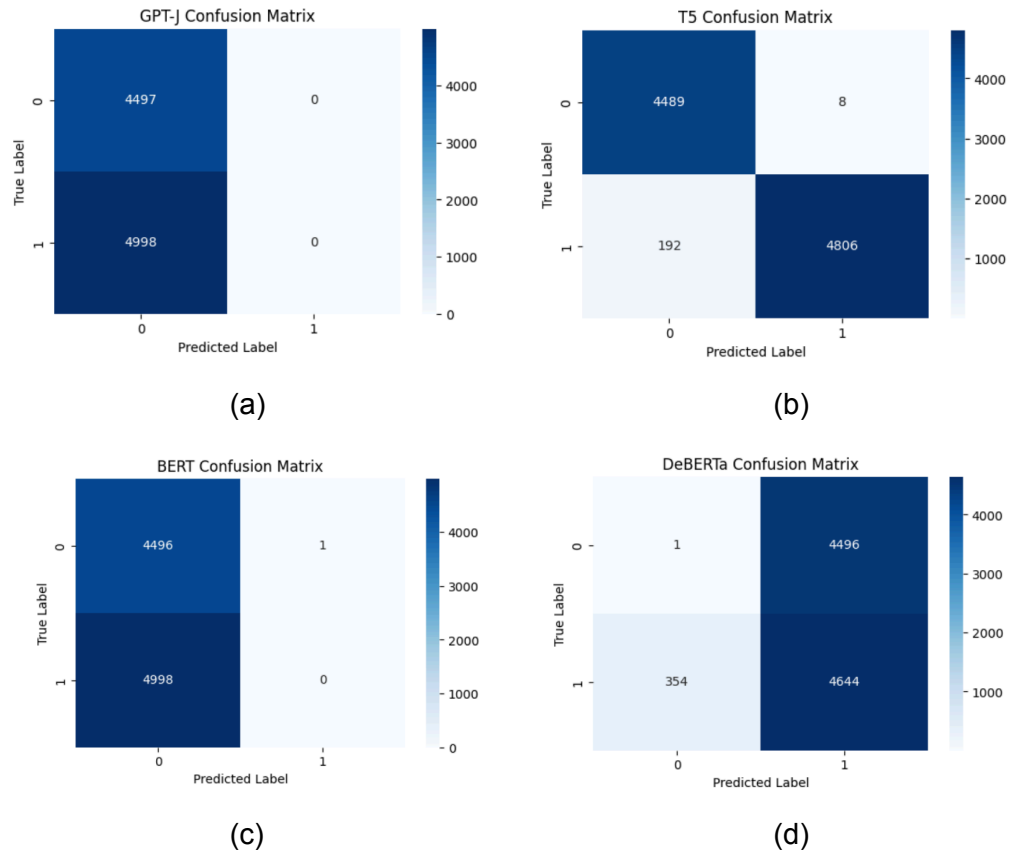


Figure 5.1 Confusion Matrix Graph Results

As illustrated in Figure 5.1, GPT-J (a) and BERT (c) both predicted almost all samples as class 0, completely failing to identify class 1. This indicates severe class bias, likely due to overfitting, poor fine-tuning, or dataset imbalance. Despite BERT's (c) design for classification, it showed the same issue as GPT-J (a), suggesting inadequate training adjustments.

DeBERTa (d), in contrast, predicted most samples as class 1, successfully capturing the minority class but at the cost of misclassifying nearly all class 0 instances. This suggests DeBERTa (d) overcompensated toward class 1, possibly due to aggressive thresholding or sensitivity to dataset skewness.

T5 (b) achieved balanced and accurate classification, with minimal misclassifications across both classes. Its strong performance reflects its flexible text-to-text architecture and diverse pre-training, enabling better generalisation and class separation compared to the other models.

5.2.2 ROC-AUC

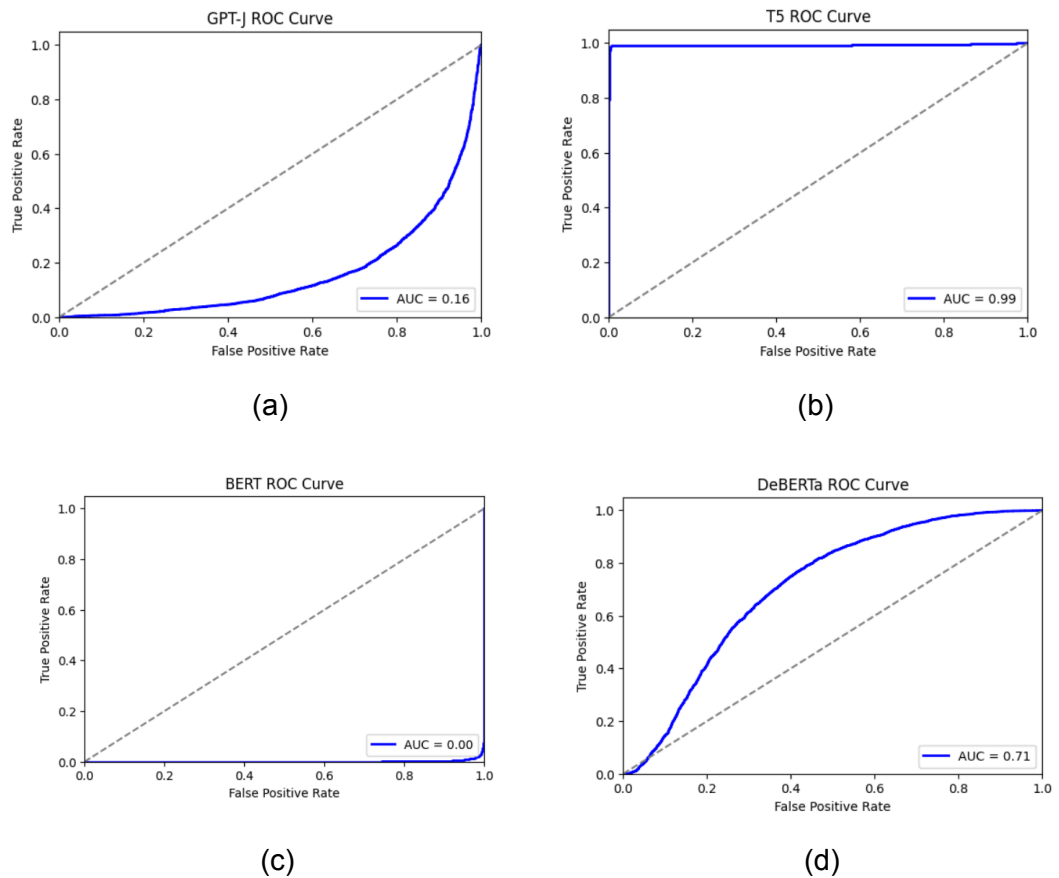


Figure 5.2 ROC-AUC Graph Results

As illustrated in Figure 5.2, GPT-J (a) shows an AUC (Area Under Curve) of 0.16, indicating extremely poor performance. Its curve lies close to the diagonal line or worse, suggesting the model performs worse than random guessing and struggles significantly to separate positive from negative classes.

BERT's (c) ROC curve is even more concerning, with an AUC of 0.00. This suggests that BERT (c) is completely unable to distinguish between the two classes. The curve remains stuck along the bottom axis, confirming that the model consistently predicts a single class regardless of the input, aligning with earlier observations from the confusion matrix.

DeBERTa (d) shows a clear improvement, achieving an AUC of 0.71. While not perfect, this AUC indicates that DeBERTa (d) has a moderate ability to discriminate between classes. The curve rises steadily above the diagonal, demonstrating that the model correctly ranks positive samples higher than negative ones most of the time, although there is still room for improvement.

T5 (b) significantly outperforms the others with an AUC of 0.99. Its ROC curve hugs the top-left corner closely, reflecting near-perfect classification ability. T5 is highly capable of distinguishing between classes with minimal false positives or false negatives, matching its strong confusion matrix performance and high accuracy.

5.3 Chapter Summary

This chapter evaluated the performance of four LLMs in detecting AI-generated text. GPT-J and BERT showed strong class bias, failing to detect AI content effectively. DeBERTa offered moderate improvement by identifying the minority class but misclassified much of the human-written text. T5 achieved the highest accuracy and balance, supported by strong F1-scores and a near-perfect ROC-AUC of 0.99. These results highlight the importance of model architecture, with T5's encoder-decoder design proving most effective. The next chapter concludes the study and outlines future enhancements for the detection system.

6.0 Contributions and Future Works

6.1 Conclusion

As a summary, T5 proves to be the best model in detecting AI generated text. This project evaluates the effectiveness of GPT-J, BERT, DeBERTa, and T5 in detecting AI-generated text. GPT-J and BERT struggled with severe class bias, while DeBERTa showed moderate improvement by better capturing minority class patterns. T5 outperformed all other models, achieving near-perfect accuracy due to its flexible text-to-text design and strong pretraining. Therefore, development of the Streamlit web application was built on T5, further demonstrating the practical application of this research. Overall, this study highlights that model architecture and training approaches significantly influencing AI text detection capabilities, providing a strong foundation for future advancements in this research domain. This dissertation has successfully met all stated aims and objectives as stated in Subsection 1.3.

6.2 Future Works

Future works can explore implementing larger parameter LLMs such as GPT-NeoX-20B to evaluate improvements in detection performance. More diverse evaluation metrics, including MCC and Precision-Recall AUC, should be applied for a fuller performance view. Metric thresholds like perplexity, entropy, and semantic coherence could be fine-tuned more strictly to enhance classification accuracy. Additionally, the Streamlit app can be expanded by adding word-level highlighting for suspected AI-generated text, improving user interpretability. Adding batch analysis functionality would increase efficiency for users handling large volumes of text. Finally, integrating ensemble methods to combine multiple model outputs could boost robustness and reduce individual model bias, leading to more consistent detection results.

6.3 Reflection

This dissertation has been a valuable learning experience, both technically and personally. It deepened my understanding of Large Language Models and AI text detection, while enhancing my skills in using tools such as Hugging Face Transformers, Google Colab Pro+, and Streamlit. I also became more proficient in applying linguistic metrics like perplexity, entropy, and semantic coherence for performance evaluation. On a personal level, the project improved my problem-solving abilities, time management, and confidence in developing end-to-end machine learning systems.

7.0 Bibliography

- [1] G. Orrù, A. Piarulli, C. Conversano, and A. Gemignani, "Human-like problem-solving abilities in large language models using ChatGPT," *Frontiers in Artificial Intelligence*, vol. 6, p. 1199350, 2023.
- [2] S. Kukreja, T. Kumar, A. Purohit, A. Dasgupta, and D. Guha, "A literature survey on open source large language models," in *Proc. 2024 7th Int. Conf. Comput. Manage. Bus.*, Jan. 2024, pp. 133–143.
- [3] M. Shao, A. Basit, R. Karri, and M. Shafique, "Survey of different large language model architectures: Trends, benchmarks, and challenges," *IEEE Access*, 2024.
- [4] Y. Guo, W. Qiu, G. Leroy, S. Wang, and T. Cohen, "Retrieval augmentation of large language models for lay language generation," *J. Biomed. Inform.*, vol. 149, p. 104580, 2024.
- [5] U. Khaliq and P. Patel, "Investigating fine-tuning of large language models for text summarisation," in *Proc. Int. Conf. Frontiers Intell. Comput.: Theory Appl.*, Singapore: Springer Nature Singapore, Jun. 2024, pp. 71–81.
- [6] A. Kostina, M. D. Dikaiakos, D. Stefanidis, and G. Pallis, "Large language models for text classification: Case study and comprehensive review," *arXiv preprint arXiv:2501.08457*, 2025.
- [7] H. L. Hofmann, G. A. Guerra, J. L. Le, A. M. Wong, G. H. Hofmann, C. K. Mayfield, and J. N. Liu, "The rapid development of artificial intelligence: GPT-4's performance on orthopedic surgery board questions," *Orthopedics*, vol. 47, no. 2, pp. E85–e89, 2024.
- [8] L. Yang, S. Xu, A. Sellergren, T. Kohlberger, Y. Zhou, I. Ktena, and D. Golden, "Advancing multimodal medical capabilities of Gemini," *arXiv preprint, arXiv:2405.03162*, 2024.
- [9] M. U. Hadi, R. Qureshi, A. Shah, M. Irfan, A. Zafar, M. B. Shaikh, and S. Mirjalili, "Large language models: A comprehensive survey of its applications, challenges, limitations, and future prospects," *Authorea Preprints*, 2023.
- [10] T. C. Callari and L. Puppione, "Can generative artificial intelligence productivity tools support workplace learning? A qualitative study on employee perceptions in a multinational corporation," *Journal of Workplace Learning*, 2025.
- [11] J. Alammam and M. Grootendorst, *Hands-on Large Language Models: Language Understanding and Generation*, O'Reilly Media, Inc., 2024.
- [12] E. Haque, *A Beginner's Guide to Large Language Models*, Enamul Haque, 2024.

- [13] J. McCarthy, "What is artificial intelligence?," Stanford University, 2007. [Online]. Available: <http://www-formal.stanford.edu/jmc/whatisai/>.
- [14] T. Amaratunga, *Understanding Large Language Models*. Thimira Amaratunga, 2023.
- [15] X. Lu, Q. Wang, and X. Liu, "Large Language Model Understands Chinese Better with Mega Tokenization," *Research Square*, June 10, 2024. DOI: [10.21203/rs.3.rs-4542532/v1](https://doi.org/10.21203/rs.3.rs-4542532/v1).
- [16] N. Hussain, A. Qasim, G. Mehak, and O. Kolesnikova, "Hybrid machine learning and deep learning approaches for insult detection in Roman Urdu text," *AI*, MDPI, 2025. [Online]. Available: <https://www.mdpi.com/2673-2688/6/2/33>.
- [17] Z. Xiong, Y. Cai, B. Hooi, N. Peng, K. W. Chang, and Z. Li, "Enhancing LLM character-level manipulation via divide and conquer," *arXiv preprint arXiv:2502.08180*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.08180>.
- [18] H. M. Nguyen, S. N. Shukla, Q. Zhang, and H. Yu, "BRIDLE: Generalized self-supervised learning with quantization," *arXiv preprint arXiv:2502.02118*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.02118>.
- [19] E. Asgari, Y. E. Kheir, and M. A. S. Javaheri, "MorphBPE: A morpho-aware tokenizer bridging linguistic complexity for efficient LLM training across morphologies," *arXiv preprint arXiv:2502.00894*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.00894>.
- [20] F. Sadat, "Studying the effect of Hindi tokenizer performance on downstream tasks," *ACL Anthology*, 2025. [Online]. Available: <https://aclanthology.org/2025.indonlp-1.5/>.
- [21] L. Beinborn and Y. Pinter, "Analyzing cognitive plausibility of subword tokenization," *arXiv preprint arXiv:2310.13348*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.13348>.
- [22] K. Batsuren, E. Vylomova, V. Dankers, T. Delgerbaatar, O. Uzan, Y. Pinter, and G. Bella, "Evaluating subword tokenization: Alien subword composition and OOV generalization challenge," *arXiv preprint arXiv:2404.13292*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.13292>.
- [23] D. Herrmannova, "Byte Pair Encoding Algorithm," *One Minute NLP*, [Online]. Available: <https://oneminutenlp.substack.com/p/byte-pair-encoding-algorithm>. [Accessed: Feb. 12, 2025].
- [24] K. Vayadande, D. R. Kale, and J. Nalavade, "Text generation & classification in NLP: A review," *Wiley Online Library*, 2024. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781394214167.ch3>.

- [25] D. Feher, "Retrofitting language models with dynamic tokenization," *University of Cambridge*, 2024. [Online]. Available: https://www.mlmi.eng.cam.ac.uk/files/2023-2024/feher_retrofitting_2024_0.pdf.
- [26] Google Research, "A fast WordPiece tokenization system," *Google Research Blog*, [Online]. Available: <https://research.google/blog/a-fast-wordpiece-tokenization-system/>. [Accessed: Feb. 18, 2025].
- [27] E. Asgari, Y. E. Kheir, and M. A. S. Javaheri, "MorphBPE: A morpho-aware tokenizer bridging linguistic complexity for efficient LLM training across morphologies," *arXiv preprint arXiv:2502.00894*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.00894>.
- [28] M. Behjati, "Discovering meaningful units from text sequences," *EPFL*, 2024. [Online]. Available: <https://infoscience.epfl.ch/bitstreams/d10b3856-0273-4271-9c52-69700d09c550/download>.
- [29] K. Sooksatra, A. R. Perez, and P. Rivas, "Resilient word-embedding alignment for BERT with character-based layer in noisy environments," *2024 International Joint Conference on NLP*, 2024. [Online]. Available: <https://www.rivas.ai/pdfs/sooksatra2024resilient.pdf>.
- [30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [31] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [32] Z. Wang, Z. Chu, T. V. Doan, S. Ni, M. Yang, and W. Zhang, "History, development, and principles of large language models: An introductory survey," *AI and Ethics*, vol. 2024, pp. 1–17, 2024.
- [33] H. Jin, W. Wei, X. Wang, W. Zhang, and Y. Wu, "Rethinking learning rate tuning in the era of large language models," in *Proc. 2023 IEEE 5th Int. Conf. Cogn. Mach. Intell. (CogMI)*, Nov. 2023, pp. 112–121.
- [34] Y. Fu, H. Peng, and T. Khot, "How does GPT obtain its ability? Tracing emergent abilities of language models to their sources," *Yao Fu's Notion*, 2022. [Online]. Available: <https://yaofu.notion.site/How-does-GPT-Obtain-its-Ability-Tracing-Emergent-Abilities-of-Language-Models-to-their-Sources-b9a57ac0fcf74f30a1ab9e3e36fa1dc1>. [Accessed: Feb. 18, 2025].

- [35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018. [Online]. Available: <https://arxiv.org/abs/1810.04805>. [Accessed: Feb. 18, 2025].
- [36] C. Wang, M. Li, and A. J. Smola, "Language models with transformers," *arXiv preprint arXiv:1904.09408*, 2019. [Online]. Available: <https://arxiv.org/abs/1904.09408>. [Accessed: Mar. 1, 2025].
- [37] T. Wang, A. Roberts, D. Hesslow, T. Le Scao, H. W. Chung, I. Beltagy, J. Launay, and C. Raffel, "What language model architecture and pretraining objective works best for zero-shot generalization?" in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2022, pp. 22964–22984. [Accessed: Mar. 1, 2025].
- [38] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [39] C. Chaka, "Detecting AI content in responses generated by ChatGPT, YouChat, and Chatsonic: The case of five AI content detection tools," *J. Appl. Learn. Teach.*, vol. 6, no. 2, pp. 94–104, 2023.
- [40] C. Chaka, "Generative AI chatbots-ChatGPT versus YouChat versus Chatsonic: Use cases of selected areas of applied English language studies," *Int. J. Learn. Teach. Educ. Res.*, vol. 22, no. 6, pp. 1–19, 2023.
- [41] E. Ofgang, "What is GPTZero? The ChatGPT detection tool explained by its creator," *Tech & Learning*, vol. 27, 2023.
- [42] T. Desk, "GPTZero to help teachers deal with ChatGPT-generated student essays," 2023.
- [43] J. Tyrrell, "ChatGPT screening: OpenAI text classifier versus GPTZero app," *TechHQ*, 2023. [Online]. Available: <https://techhq.com/2023/02/chatgpt-screening-openai-text-classifier-versusgptzero-a-pp/>. [Accessed: Mar. 3, 2025].
- [44] C. Chaka, "Stylised-facts view of fourth industrial revolution technologies impacting digital learning and workplace environments: ChatGPT and critical reflections," *Frontiers in Education*, vol. 8, 2023. [Online]. Available: <https://doi.org/10.3389/feduc.2023.1150499>. [Accessed: Mar. 3, 2025].
- [45] E. Eliaçık, "AI Text Classifier: OpenAI's ChatGPT detector indicates AI-generated text," *Dataconomy*, 2023. [Online]. Available: <https://dataconomy.com/2023/02/openai-ai-text-classifier-chatgpt-detector/>. [Accessed: Mar. 3, 2025].

- [46] M. Ismail, "OpenAI releases AI classifier to check for ChatGPT-written articles, still needs work," *TechNave*, 2023. [Online]. Available: <https://technave.com/gadget/OpenAI-releases-AI-classifier-to-check-for-Chat-GPT-written-articles-still-needs-work-33313.html>. [Accessed: Mar. 3, 2025].
- [47] J. H. Kirchner, L. Ahmad, S. Aaronson, and J. Leike, "New AI classifier for indicating AI-written text," *OpenAI*, 2023. [Online]. Available: <https://openai.com/blog/new-ai-classifier-for-indicating-ai-written-text>. [Accessed: Mar. 3, 2025].
- [48] O. Spotlight, "Best AI content detection tools: Free ChatGPT output detector," *Outlook India*, 2023.
- [49] Help Center, "AI content detector," *Writer.com*, 2023. [Online]. Available: <https://support.writer.com/article/205-ai-content-detector>. [Accessed: Mar. 3, 2025].
- [50] H. Lim, "Content detection tools to tell if content is written by ChatGPT," *HongKiat*, vol. 5, 2023.
- [51] Chrome, "AI content detector - Copyleaks," *Chrome Web Store*, 2023. [Online]. Available: <https://chrome.google.com/webstore/detail/ai-content-detector-copy/gplcmncpklkdjiccbknjjkoidpgkckad>. [Accessed: Mar. 3, 2025].
- [52] Copyleaks, "AI Content Detector," *Copyleaks*, [Online]. Available: <https://copyleaks.com/ai-content-detector>. [Accessed: Mar. 17, 2025].
- [53] D. Weber-Wulff, A. Anohina-Naumeca, S. Bjelobaba, T. Foltýnek, J. Guerrero-Dib, O. Popoola, P. Šigut, and L. Waddington, "Testing of detection tools for AI-generated text," *arXiv preprint arXiv:2306.15666*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2306.15666>. [Accessed: Mar. 3, 2025].
- [54] K. Wiggers, "Most sites claiming to catch AI-written text fail spectacularly," *TechCrunch*, 2023. [Online]. Available: <https://techcrunch.com/2023/02/16/most-sites-claiming-to-catch-ai-written-text-fail-spectacularly/>. [Accessed: Mar. 4, 2025].
- [55] T. Aremu, "Unlocking Pandora's box: Unveiling the elusive realm of AI text detection," 2023. [Online]. Available: https://papers.ssrn.com/sol3/Delivery.cfm/SSRN_ID4470719_code5947956.pdf?abstractid=4470719&mirid=1. [Accessed: Mar. 4, 2025].
- [56] Z. Xu and P. Zhu, "Using BERT-based textual analysis to design a smarter classroom mode for computer teaching in higher education institutions," *International Journal of Emerging Technologies in Learning (iJET)*, vol. 18, no. 19, pp. 114–127, 2023. [Online]. Available: <https://doi.org/10.3991/ijet.v18i19.42483>. [Accessed: Mar. 10, 2025].

- [57] J. C. Timoneda and S. V. Vera, "BERT, RoBERTa or DeBERTa? Comparing performance across transformers models in political science text," *The Journal of Politics*, vol. 87, no. 1, pp. 000–000, 2025.
- [58] Hugging Face, "GPT-J," *Transformers Documentation*, [Online]. Available: https://huggingface.co/docs/transformers/en/model_doc/gptj. [Accessed: Mar. 17, 2025].
- [59] N. Vassilieva, "Cerebras makes it easy to harness the predictive power of GPT-J," *Cerebras Systems*, Jun. 22, 2022. [Online]. Available: <https://cerebras.ai/blog/cerebras-makes-it-easy-to-harness-the-predictive-power-of-gpt-j>. [Accessed: Mar. 17, 2025].
- [60] K. Gao, S. He, Z. He, J. Lin, Q. Pei, J. Shao, and W. Zhang, "Examining user-friendly and open-sourced large GPT models: A survey on language, multimodal, and scientific GPT models," *arXiv preprint arXiv:2308.14149*, Aug. 2023. [Online]. Available: <https://arxiv.org/abs/2308.14149>. [Accessed: Mar. 17, 2025].
- [61] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, May 24, 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>. [Accessed: Mar. 17, 2025].
- [62] Coursera, "BERT Model: What It Is and How It Works," *Coursera Articles*, [Online]. Available: <https://www.coursera.org/articles/bert-model>. [Accessed: Mar. 17, 2025].
- [63] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://arxiv.org/abs/2006.03654>. [Accessed: Mar. 17, 2025].
- [64] G. Gupta, "Understanding the T5 model: A comprehensive guide," *Medium*, Jan. 2024. [Online]. Available: https://medium.com/@gagangupta_82781/understanding-the-t5-model-a-comprehensive-guide-b4d5c02c234b. [Accessed: Mar. 17, 2025].
- [65] I. Luden, M. Giulianelli, and R. Fernández, "Beyond perplexity: Examining temporal generalization in large language models via definition generation," *Computational Linguistics in the Netherlands Journal*, vol. 13, pp. 205–232, 2024.
- [66] Y. Zhang, X. Chen, P. Jin, and Y. Du, "Exploring context's diversity to improve neural language model," in *Proc. 2019 Int. Conf. Asian Language Processing (IALP)*, Nov. 2019, pp. 419–423. IEEE.
- [67] S. S. Hurtado, T. Mullen, T. Onodera, and P. Sheridan, "A statistical significance testing approach for measuring term burstiness with applications to domain-specific terminology extraction," *arXiv preprint arXiv:2310.15790*, 2023.

- [68] N. K. Jha and B. Reagen, "Entropy-guided attention for private LLMs," *arXiv preprint arXiv:2501.03489*, 2025.
- [69] M. Sartor, F. Dell'Orletta, and G. Venturi, "Coherence evaluation in Italian language models," in *Proc. 8th Workshop on Natural Language for Artificial Intelligence (NL4AI 2024), co-located with the 23rd Int. Conf. of the Italian Association for Artificial Intelligence (AIIA 2024)**, CEUR-WS.org, 2024.
- [70] A. Goswami, G. Kaur, S. Tayal, A. Verma, and M. Verma, "Analyzing the efficacy of Deep Learning and Transformer models in classifying Human and LLM-Generated Text," in *Proc. 8th Int. Conf. Comput., Commun., Control Autom. (ICCUBEA)*, Pune, India, Aug. 2024, pp. 1–5.

8.0 Appendix

Appendix A: Pseudocode for creating Human-Generated Text Dataset

```
BEGIN

# Import necessary modules
IMPORT requests, BeautifulSoup, pandas, datetime, tqdm, nltk

# Download and setup NLTK tokenizer
DOWNLOAD 'punkt' (if not available)
IMPORT sent_tokenize FROM nltk.tokenize

# Function to split text into sentences
FUNCTION split_into_sentences(text):
    RETURN sent_tokenize(text)

# Function to remove full stops at the end of sentences
FUNCTION remove_full_stop(sentences):
    FOR each sentence in sentences:
        REMOVE trailing '.' (if present)
    RETURN modified sentences

# Function to scrape content from a given URL
FUNCTION scrape_content(url, content_type=None):
    TRY:
        response ← Fetch URL with 'User-Agent' header
        CHECK response status
    CATCH request error:
        PRINT "Error fetching URL"
        RETURN empty list

# Parse HTML content
soup ← Parse HTML using BeautifulSoup

IF content_type == 'paper':
    content ← Extract text from 'abstract' class
ELSE:
    content ← Concatenate all paragraphs' text

IF content is empty:
    PRINT "Warning: No content found"
    RETURN empty list

# Process text into sentences
sentences ← split_into_sentences(content)
cleaned_sentences ← remove_full_stop(sentences)
```

```
    RETURN [{ 'text': sentence, 'generated': 0, 'source': url } FOR each  
sentence in cleaned_sentences]
```

```
# Function to scrape specific research article links
```

```
FUNCTION scrape_research_links(research_links):
```

```
    PRINT "Scraping research articles..."
```

```
    results ← []
```

```
    FOR each url in research_links:
```

```
        TRY:
```

```
            data ← scrape_content(url, content_type='paper')
```

```
            ADD data to results
```

```
        CATCH error:
```

```
            PRINT "Failed to scrape research article"
```

```
    RETURN results
```

```
# MAIN FUNCTION
```

```
FUNCTION main():
```

```
    # Define list of URLs to scrape
```

```
    urls ← LIST OF NEWS, BUSINESS, TECHNOLOGY, AND OTHER  
SOURCES
```

```
    research_links ← LIST OF RESEARCH ARTICLE URLs
```

```
# Initialize storage for scraped data
```

```
scraped_data ← []
```

```
research_data ← []
```

```
# Scrape general URLs
```

```
FOR each url in urls:
```

```
    TRY:
```

```
        content_type ← 'paper' IF 'pmc' in url ELSE 'news'
```

```
        data ← scrape_content(url, content_type)
```

```
        ADD data to scraped_data
```

```
    CATCH error:
```

```
        PRINT "Failed to scrape URL"
```

```
# Scrape research articles
```

```
research_data ← scrape_research_links(research_links)
```

```
# Combine data
```

```
all_data ← scraped_data + research_data
```

```
# Convert to DataFrame
```

```
IF all_data is NOT EMPTY:
```

```
    df ← Convert all_data to DataFrame
```

```
    REMOVE duplicate 'text' entries
```

```
    REMOVE empty or NULL values
```

```

# Save dataset without source column
df_no_source ← df with only ['text', 'generated']
filename_no_source ← "human_generated_data_TIMESTAMP.csv"
SAVE df_no_source to CSV

# Save dataset with source column
filename_with_source ←
"human_generated_data_with_sources_TIMESTAMP.csv"
SAVE df to CSV

PRINT "Datasets saved successfully"
ELSE:
PRINT "No valid data collected. Exiting."

# Run the script
IF __name__ == "__main__":
CALL main()

END

```

Appendix B: Pseudocode for creating AI-Generated Text Dataset

```

BEGIN

# Import necessary modules
IMPORT torch, transformers, pandas, csv, datetime, logging

# Configure logging
SET logging level to INFO
DEFINE logging format as "[Timestamp] - [Level] - [Message]"

# Define Model Configuration
DEFINE models AS a list containing:
- GPT-J Model with tokenizer, max_length=400, temperature=0.7
- GPT-2 Model with tokenizer, max_length=300, temperature=0.8
- T5 Model with tokenizer, max_length=200, temperature=0.7

# Function to Generate Text using AI Model
FUNCTION generate_text(model, tokenizer, prompt, max_length,
temperature):
TRY:
Encode the prompt using tokenizer
Generate output using model with parameters:
- max_length
- temperature
- top_k=50

```

```

        - top_p=0.95
        - do_sample=True
    Decode and return generated text
CATCH error:
    LOG "Error generating text for prompt"
    RETURN None

# Function to Save Generated Texts to CSV
FUNCTION save_to_csv(output_data):
    CREATE a filename with current timestamp
    OPEN a CSV file in write mode
    WRITE headers: "text", "generated"
    WRITE rows of output_data
    LOG "Generated texts saved successfully"

# Function to Generate Unique AI Texts
FUNCTION generate_unique_ai_texts(limit):
    INITIALIZE output_data as empty list
    INITIALIZE generated_texts as empty set
    SET count = 0

    FOR each category in categories:
        FOR each prompt in category:
            IF count >= limit THEN BREAK

            FOR each model in models:
                IF count >= limit THEN BREAK

                LOG "Generating text with MODEL_NAME for PROMPT"
                CALL generate_text(model, tokenizer, prompt, max_length,
temperature)

                IF generated_text is UNIQUE:
                    ADD generated_text to output_data
                    ADD generated_text to generated_texts
                    INCREMENT count

    CALL save_to_csv(output_data)

# Define Categories with Predefined Prompts
DEFINE categories AS dictionary containing:
    - College: List of prompts
    - Electoral: List of prompts
    - Help: List of prompts
    - Important: List of prompts
    - Like: List of prompts
    - Technology: List of prompts
    - Health: List of prompts

```

```

- Environment: List of prompts
- Education: List of prompts
- Society: List of prompts
- Business: List of prompts
- Science: List of prompts
- Public Health Policy: List of prompts
- Healthcare Disparities: List of prompts
- Others: List of prompts

# Execute AI Text Generation
IF script is run as main:
    SET max_unique_texts = 10
    CALL generate_unique_ai_texts(max_unique_texts)

END

```

Appendix C: Pseudocode for combining Human & AI Datasets

```

BEGIN

# Import necessary library
IMPORT pandas AS pd

# Define file paths
SET ai_generated_path = "Path to AI-generated dataset"
SET human_generated_path = "Path to Human-generated dataset"

# Load datasets into DataFrames
ai_df = READ CSV ai_generated_path
human_df = READ CSV human_generated_path

# Ensure both datasets have the same columns
IF columns of ai_df are NOT equal to columns of human_df THEN:
    PRINT "Warning: Column mismatch detected. Standardizing columns."
    common_columns = FIND common columns between ai_df and
human_df
    ai_df = SELECT common_columns from ai_df
    human_df = SELECT common_columns from human_df

# Combine both datasets
combined_df = CONCATENATE ai_df and human_df, RESET index

# Handle missing values
IF combined_df has missing values THEN:
    PRINT "Missing values detected. Filling with placeholder."
    FILL missing values with "Unknown"

```

```

# Ensure data types are consistent
FOR each column in combined_df:
    IF data type of ai_df[column] != data type of human_df[column] THEN:
        PRINT "Type mismatch in column. Converting to string."
        CONVERT column data type to STRING

# Remove duplicate rows
DROP duplicate rows from combined_df

# Shuffle the dataset
shuffled_df = RANDOMLY SHUFFLE rows of combined_df (set seed = 42)

# Save shuffled dataset to CSV
output_path = "Path to save final dataset"
SAVE shuffled_df AS CSV at output_path (without index)

PRINT "Shuffled combined dataset saved to: output_path"

END

```

Appendix D: Pseudocode for dataset analysis by GPT-J

```

BEGIN

# Import necessary libraries
IMPORT torch
IMPORT numpy AS np
IMPORT transformers (AutoTokenizer, AutoModelForCausalLM)
IMPORT nltk
IMPORT sklearn.preprocessing (LabelEncoder)
IMPORT sklearn.metrics (accuracy_score, classification_report,
confusion_matrix, roc_curve, auc)
IMPORT pandas AS pd
IMPORT matplotlib.pyplot AS plt
IMPORT seaborn AS sns

# Download necessary nltk resources
DOWNLOAD nltk 'punkt'

# Define file path
SET uploaded_file_path TO '/content/drive/MyDrive/FYP Dataset
Folder/FYP_Dataset.csv'

# Load dataset
LOAD data FROM uploaded_file_path AS CSV

```



```

# Extract text and labels
SET texts TO data['text']
SET labels TO data['generated']

# Encode labels using LabelEncoder
INITIALIZE label_encoder AS LabelEncoder
SET labels_encoded TO label_encoder.fit_transform(labels)

# Load GPT-J model and tokenizer
LOAD gptj_tokenizer FROM pretrained 'EleutherAI/gpt-j-6B'
LOAD gptj_model FROM pretrained 'EleutherAI/gpt-j-6B'

# Define text cleaning function
FUNCTION clean_text(text):
    REPLACE '\n' AND '\r' WITH ' ' IN text
    REMOVE EXTRA SPACES IN text
    RETURN cleaned text

# Define function to truncate input sequence for model
FUNCTION truncate_sequence(tokenizer, text, max_length):
    TOKENIZE text USING tokenizer WITH truncation=True,
max_length=max_length, return_tensors='pt'
    RETURN tokens

# Define function to calculate GPT-J perplexity and burstiness
FUNCTION calculate_gptj_perplexity_and_burstiness(text):
    CLEAN text USING clean_text()
    TOKENIZE text USING truncate_sequence() WITH max length from
gptj_model
    WITH NO GRADIENT:
        GET outputs FROM gptj_model USING tokens
        EXTRACT loss FROM outputs
        COMPUTE perplexity AS exp(loss)
        COMPUTE logits FROM outputs
        COMPUTE probabilities USING softmax(logits)
        COMPUTE log probabilities
        COMPUTE burstiness AS variance of log probabilities
        RETURN perplexity, burstiness

# Initialize prediction lists
INITIALIZE gptj_predictions AS empty list
INITIALIZE gptj_scores AS empty list

# Set classification thresholds
SET GPTJ_PERPLEXITY_THRESHOLD TO 50
SET GPTJ_BURSTINESS_THRESHOLD TO 0.5

```

```

# Loop through texts and make predictions
FOR EACH text IN texts:
    COMPUTE perplexity_gptj, burstiness_gptj USING
calculate_gptj_perplexity_and_burstiness(text)
    IF perplexity_gptj < GPTJ_PERPLEXITY_THRESHOLD AND
burstiness_gptj < GPTJ_BURSTINESS_THRESHOLD:
        APPEND 1 TO gptj_predictions
    ELSE:
        APPEND 0 TO gptj_predictions
    APPEND perplexity_gptj TO gptj_scores

# Convert numerical predictions to original labels
SET gptj_labels TO label_encoder.inverse_transform(gptj_predictions)

# Print evaluation results
PRINT "=====
PRINT "GPT-J Evaluation"
PRINT "=====
PRINT "Accuracy:", accuracy_score(labels, gptj_labels)
PRINT "Classification Report:", classification_report(labels, gptj_labels,
zero_division=0)

# Convert labels to binary format
SET labels_binary TO label_encoder.transform(labels)

# Define function to plot confusion matrix
FUNCTION plot_confusion_matrix(y_true, y_pred, title):
    COMPUTE cm AS confusion_matrix(y_true, y_pred)
    PLOT heatmap USING seaborn WITH annotations and labels
    SET title, x-label, y-label
    SHOW plot

# Define function to plot ROC curve
FUNCTION plot_roc_curve(y_true, y_scores, title):
    COMPUTE fpr, tpr USING roc_curve(y_true, y_scores)
    COMPUTE roc_auc USING auc(fpr, tpr)
    PLOT ROC curve
    SHOW plot

# Plot confusion matrix
CALL plot_confusion_matrix(labels_binary, gptj_predictions, "GPT-J
Confusion Matrix")

# Normalize scores for ROC curve
INITIALIZE MinMaxScaler
SCALE gptj_scores TO range (0,1)

# Compute AUC score

```

```

COMPUTE fpr, tpr USING roc_curve(labels_binary, gptj_scores)
COMPUTE roc_auc USING auc(fpr, tpr)

# Check if AUC is inverted
IF roc_auc < 0.5:
    PRINT "ROC curve is inverted. Negating scores to fix."
    NEGATE gptj_scores

# Plot ROC curve
CALL plot_roc_curve(labels_binary, gptj_scores, "GPT-J ROC Curve")

END

```

Appendix E: Pseudocode for dataset analysis by BERT

```

BEGIN

# Import necessary libraries
IMPORT torch, numpy, transformers (BertTokenizer, BertForMaskedLM)
IMPORT nltk, pandas, sklearn (LabelEncoder, metrics)
IMPORT matplotlib, seaborn

# Download necessary NLTK resources
DOWNLOAD nltk 'punkt'

# Define file path and load dataset
SET uploaded_file_path = "Path to dataset"
LOAD data FROM uploaded_file_path AS CSV

# Extract text and labels
SET texts = data['text']
SET labels = data['generated']

# Encode labels
INITIALIZE label_encoder AS LabelEncoder
SET labels_encoded = label_encoder.fit_transform(labels)

# Load BERT model and tokenizer
LOAD bert_tokenizer FROM pretrained 'bert-base-uncased'
LOAD bert_model FROM pretrained 'bert-base-uncased'

# Define function to clean text
FUNCTION clean_text(text):
    REMOVE newlines and extra spaces
    RETURN cleaned text

```

```

# Define function to truncate input sequence
FUNCTION truncate_sequence(tokenizer, text, max_length):
    TOKENIZE text WITH truncation, max_length, and return_tensors='pt'
    RETURN tokens

# Define function to calculate BERT entropy
FUNCTION calculate_bert_entropy(text):
    CLEAN text USING clean_text()
    TOKENIZE text USING truncate_sequence() WITH max length from
bert_model
    WITH NO GRADIENT:
        GET model outputs
        COMPUTE softmax probabilities and log probabilities
        COMPUTE entropy AS -SUM(probs * log_probs)
    RETURN entropy

# Initialize prediction lists
INITIALIZE bert_predictions AS empty list
INITIALIZE bert_scores AS empty list

# Set entropy threshold
SET BERT_THRESHOLD = 3.8

# Evaluate each text
FOR EACH text IN texts:
    COMPUTE entropy_bert USING calculate_bert_entropy(text)
    IF entropy_bert > BERT_THRESHOLD:
        APPEND 1 TO bert_predictions
    ELSE:
        APPEND 0 TO bert_predictions
    APPEND entropy_bert TO bert_scores

# Convert numerical predictions to original labels
SET bert_labels = label_encoder.inverse_transform(bert_predictions)

# Print evaluation results
PRINT "=====
PRINT "BERT Evaluation"
PRINT "=====
PRINT "Accuracy:", accuracy_score(labels, bert_labels)
PRINT "Classification Report:", classification_report(labels, bert_labels,
zero_division=0)

# Convert labels to binary format
SET labels_binary = label_encoder.transform(labels)

# Define function to plot confusion matrix
FUNCTION plot_confusion_matrix(y_true, y_pred, title):

```

```

    COMPUTE cm = confusion_matrix(y_true, y_pred)
    PLOT heatmap with labels and annotations
    SHOW plot

# Define function to plot ROC curve
FUNCTION plot_roc_curve(y_true, y_scores, title):
    COMPUTE fpr, tpr USING roc_curve(y_true, y_scores)
    COMPUTE roc_auc USING auc(fpr, tpr)
    PLOT ROC curve
    SHOW plot

# Plot confusion matrix
CALL plot_confusion_matrix(labels_binary, bert_predictions, "BERT
Confusion Matrix")

# Normalize scores for ROC curve
INITIALIZE MinMaxScaler
SCALE bert_scores TO range (0,1)

# Compute AUC score
COMPUTE fpr, tpr USING roc_curve(labels_binary, bert_scores)
COMPUTE roc_auc USING auc(fpr, tpr)

# Check and correct inverted AUC score
IF roc_auc < 0.5:
    PRINT "ROC curve is inverted. Negating scores to fix."
    NEGATE bert_scores

# Plot ROC curve
CALL plot_roc_curve(labels_binary, bert_scores, "BERT ROC Curve")

END

```

Appendix F: Pseudocode for dataset analysis by DeBERTa

```

BEGIN

# Import necessary libraries
IMPORT torch, numpy, transformers (DebertaV2Tokenizer,
DebertaV2ForMaskedLM)
IMPORT nltk, pandas, sklearn (LabelEncoder, metrics)
IMPORT matplotlib, seaborn

# Download necessary NLTK resources
DOWNLOAD nltk 'punkt'

```

```

# Define file path and load dataset
SET uploaded_file_path = "Path to dataset"
LOAD data FROM uploaded_file_path AS CSV

# Extract text and labels
SET texts = data['text']
SET labels = data['generated']

# Encode labels
INITIALIZE label_encoder AS LabelEncoder
SET labels_encoded = label_encoder.fit_transform(labels)

# Load DeBERTa model and tokenizer
LOAD deberta_tokenizer FROM pretrained 'microsoft/deberta-v2-xlarge'
LOAD deberta_model FROM pretrained 'microsoft/deberta-v2-xlarge' WITH
output_hidden_states=True

# Define function to clean text
FUNCTION clean_text(text):
  REMOVE newlines and extra spaces
  RETURN cleaned text

# Define function to truncate input sequence
FUNCTION truncate_sequence(tokenizer, text, max_length):
  TOKENIZE text WITH truncation, max_length, and return_tensors='pt'
  RETURN tokens

# Define function to calculate DeBERTa entropy and semantic coherence
FUNCTION calculate_deberta_entropy_and_semantic_coherence(text):
  CLEAN text USING clean_text()
  TOKENIZE text USING truncate_sequence() WITH max length from
deberta_model
  WITH NO GRADIENT:
    GET model outputs
    COMPUTE softmax probabilities and log probabilities
    COMPUTE entropy AS -SUM(probs * log_probs)

  EXTRACT last hidden state
  COMPUTE cosine similarity BETWEEN adjacent hidden states
  COMPUTE semantic coherence AS MEAN(cosine similarities)

  RETURN entropy, semantic_coherence

# Initialize prediction lists
INITIALIZE deberta_predictions AS empty list
INITIALIZE deberta_scores AS empty list

# Set classification thresholds

```

```

SET DEBERTA_ENTROPY_THRESHOLD = 2
SET DEBERTA_COHERENCE_THRESHOLD = 0.8

# Evaluate each text
FOR EACH text IN texts:
    COMPUTE entropy_deberta, coherence_deberta USING
calculate_deberta_entropy_and_semantic_coherence(text)
    IF entropy_deberta > DEBERTA_ENTROPY_THRESHOLD AND
coherence_deberta < DEBERTA_COHERENCE_THRESHOLD:
        APPEND 1 TO deberta_predictions
    ELSE:
        APPEND 0 TO deberta_predictions
    APPEND entropy_deberta TO deberta_scores

# Convert numerical predictions to original labels
SET deberta_labels =
label_encoder.inverse_transform(deberta_predictions)

# Print evaluation results
PRINT "=====
PRINT "DeBERTa Evaluation"
PRINT "=====
PRINT "Accuracy:", accuracy_score(labels, deberta_labels)
PRINT "Classification Report:", classification_report(labels, deberta_labels,
zero_division=0)

# Convert labels to binary format
SET labels_binary = label_encoder.transform(labels)

# Define function to plot confusion matrix
FUNCTION plot_confusion_matrix(y_true, y_pred, title):
    COMPUTE cm = confusion_matrix(y_true, y_pred)
    PLOT heatmap with labels and annotations
    SHOW plot

# Define function to plot ROC curve
FUNCTION plot_roc_curve(y_true, y_scores, title):
    COMPUTE fpr, tpr USING roc_curve(y_true, y_scores)
    COMPUTE roc_auc USING auc(fpr, tpr)
    PLOT ROC curve
    SHOW plot

# Plot confusion matrix
CALL plot_confusion_matrix(labels_binary, deberta_predictions, "DeBERTa
Confusion Matrix")

# Normalize scores for ROC curve
INITIALIZE MinMaxScaler

```

```

SCALE deberta_scores TO range (0,1)

# Compute AUC score
COMPUTE fpr, tpr USING roc_curve(labels_binary, deberta_scores)
COMPUTE roc_auc USING auc(fpr, tpr)

# Check and correct inverted AUC score
IF roc_auc < 0.5:
    PRINT "ROC curve is inverted. Negating scores to fix."
    NEGATE deberta_scores

# Plot ROC curve
CALL plot_roc_curve(labels_binary, deberta_scores, "DeBERTa ROC
Curve")

END

```

Appendix G: Pseudocode for dataset analysis by T5

```

BEGIN

# Import necessary libraries
IMPORT torch
IMPORT numpy AS np
IMPORT transformers (T5Tokenizer, T5ForConditionalGeneration)
IMPORT nltk
IMPORT sklearn.preprocessing (LabelEncoder)
IMPORT sklearn.metrics (accuracy_score, classification_report,
confusion_matrix, roc_curve, auc)
IMPORT pandas AS pd
IMPORT matplotlib.pyplot AS plt
IMPORT seaborn AS sns

# Download necessary nltk resources
DOWNLOAD nltk 'punkt'

# Define file path
SET uploaded_file_path TO '/content/drive/MyDrive/FYP Dataset
Folder/FYP_Dataset.csv'

# Load dataset
LOAD data FROM uploaded_file_path AS CSV

# Extract text and labels
SET texts TO data['text']
SET labels TO data['generated']

```



```

# Encode labels using LabelEncoder
INITIALIZE label_encoder AS LabelEncoder
SET labels_encoded TO label_encoder.fit_transform(labels)

# Load T5 model and tokenizer
LOAD t5_tokenizer FROM pretrained 't5-small'
LOAD t5_model FROM pretrained 't5-small'

# Define text cleaning function
FUNCTION clean_text(text):
  REPLACE '\n' AND '\r' WITH ' ' IN text
  REMOVE EXTRA SPACES IN text
  RETURN cleaned text

# Define function to truncate input sequence for model
FUNCTION truncate_sequence(tokenizer, text, max_length):
  TOKENIZE text USING tokenizer WITH truncation=True,
max_length=max_length, return_tensors='pt'
  RETURN tokens

# Define function to calculate T5 perplexity
FUNCTION calculate_t5_perplexity(text):
  CLEAN text USING clean_text()
  TOKENIZE text USING truncate_sequence() WITH max length from
t5_model
  WITH NO GRADIENT:
    GET outputs FROM t5_model USING input_ids
    EXTRACT loss FROM outputs
    COMPUTE perplexity AS exp(loss)
  RETURN perplexity

# Initialize prediction lists
INITIALIZE t5_predictions AS empty list
INITIALIZE t5_scores AS empty list

# Set threshold for classification
SET T5_THRESHOLD TO 150

# Loop through texts and make predictions
FOR EACH text IN texts:
  COMPUTE perplexity_t5 USING calculate_t5_perplexity(text)
  IF perplexity_t5 > T5_THRESHOLD:
    APPEND 1 TO t5_predictions
  ELSE:
    APPEND 0 TO t5_predictions
  APPEND perplexity_t5 TO t5_scores

```

```

# Convert numerical predictions to original labels
SET t5_labels TO label_encoder.inverse_transform(t5_predictions)

# Print evaluation results
PRINT "=====
PRINT "T5 Evaluation"
PRINT "=====
PRINT "Accuracy:", accuracy_score(labels, t5_labels)
PRINT "Classification Report:", classification_report(labels, t5_labels,
zero_division=0)

# Convert labels to binary format
SET labels_binary TO label_encoder.transform(labels)

# Define function to plot confusion matrix
FUNCTION plot_confusion_matrix(y_true, y_pred, title):
    COMPUTE cm AS confusion_matrix(y_true, y_pred)
    PLOT heatmap USING seaborn WITH annotations and labels
    SET title, x-label, y-label
    SHOW plot

# Define function to plot ROC curve
FUNCTION plot_roc_curve(y_true, y_scores, title):
    COMPUTE fpr, tpr USING roc_curve(y_true, y_scores)
    COMPUTE roc_auc USING auc(fpr, tpr)
    PLOT ROC curve
    SHOW plot

# Plot confusion matrix
CALL plot_confusion_matrix(labels_binary, t5_predictions, "T5 Confusion
Matrix")

# Normalize scores for ROC curve
INITIALIZE MinMaxScaler
SCALE t5_scores TO range (0,1)

# Compute AUC score
COMPUTE fpr, tpr USING roc_curve(labels_binary, t5_scores)
COMPUTE roc_auc USING auc(fpr, tpr)

# Check if AUC is inverted
IF roc_auc < 0.5:
    PRINT "ROC curve is inverted. Negating scores to fix."
    NEGATE t5_scores

# Plot ROC curve
CALL plot_roc_curve(labels_binary, t5_scores, "T5 ROC Curve")

```

END

Appendix H: Pseudocode for Main.py (Streamlit App)

```
BEGIN

# Import necessary libraries
IMPORT streamlit, streamlit_option_menu, Chat_History,
plagiarism_detector, fitz (PyMuPDF), time

# Set up Streamlit page
SET page layout to "wide"
SET page title to "Text-minator"

# Initialize session state variables
IF 'chat_history' NOT in session_state:
    SET session_state['chat_history'] = []

IF 'model' NOT in session_state:
    SET session_state['model'] = 'GPT-2'

IF 'uploaded_text' NOT in session_state:
    SET session_state['uploaded_text'] = ""

# Function to read text from a PDF file
FUNCTION read_pdf(file, progress_bar):
    OPEN PDF file using fitz
    INITIALIZE text as an empty string
    GET total number of pages
    FOR each page in the PDF:
        EXTRACT text
        UPDATE progress bar
    RETURN extracted text

# Function to read text from a TXT file
FUNCTION read_txt(file, progress_bar):
    INITIALIZE total_size, chunk_size, bytes_read, and text as empty string
    WHILE bytes_read < total_size:
        READ chunk from file
        DECODE and append to text
        UPDATE progress bar
    RETURN extracted text

# Main application function
FUNCTION main():
    DISPLAY "Text-minator: AI Plagiarism Detector" as title
```

```

DISPLAY "Select model for analysis"

# Model selection menu
SET model = option_menu(
    options=["GPT-J", "DeBERTa", "T5", "BERT"],
    icons=["robot", "book", "bar-chart-line", "graph-up"],
    default_index=0,
    orientation="horizontal"
)
UPDATE session_state['model'] with selected model

# Text input area
SET text_area = DISPLAY text input box (default:
session_state['uploaded_text'])

# File uploader for .TXT or .PDF
SET uploaded_file = DISPLAY file uploader

# Process uploaded file
IF uploaded_file is NOT None:
    CREATE progress bar
    IF file is TXT:
        SET uploaded_text = CALL read_txt(uploaded_file, progress_bar)
    ELSE IF file is PDF:
        SET uploaded_text = CALL read_pdf(uploaded_file, progress_bar)
    CLEAR progress bar
    UPDATE session_state['uploaded_text'] with extracted text
    UPDATE text_area with uploaded_text

# Analyze button
IF text_area is NOT empty AND "Analyze" button is clicked:
    SET text_to_analyze = CLEAN text_area
    INITIALIZE result_entry with 'text' and selected 'model'

# Model-based text analysis
IF selected model is "GPT-J":
    CALL plagiarism_detector.display_results_gpt2(text_to_analyze)
    IF result is NOT None:
        EXTRACT perplexity and burstiness
        UPDATE result_entry with values
        SET result_label to 1 IF perplexity > 8000 AND burstiness > 0.2
ELSE 0

ELSE IF selected model is "DeBERTa":
    CALL plagiarism_detector.display_results_deberta(text_to_analyze)
    IF result is NOT None:
        EXTRACT entropy and semantic_coherence
        UPDATE result_entry with values

```

```

        SET result_label to 1 IF entropy > 4.5 AND semantic_coherence
        < 0.6 ELSE 0

    ELSE IF selected model is "T5":
        CALL plagiarism_detector.display_results_t5(text_to_analyze)
        IF result is NOT None:
            EXTRACT perplexity
            UPDATE result_entry
            SET result_label to 1 IF perplexity > 150 ELSE 0

    ELSE IF selected model is "BERT":
        CALL plagiarism_detector.display_results_bert(text_to_analyze)
        IF result is NOT None:
            EXTRACT entropy
            UPDATE result_entry
            SET result_label to 1 IF entropy > 3.8 ELSE 0

    # Store analysis in chat history
    APPEND result_entry to session_state['chat_history']
    DISPLAY success message "Analysis complete"

# MultiApp class for navigation
CLASS MultiApp:
    FUNCTION run():
        DISPLAY sidebar menu with ["Home", "Chat History"]
        IF "Home" selected:
            CALL main()
        ELSE IF "Chat History" selected:
            CALL Chat_History.app()

# Run the application
IF script is executed as main:
    CREATE instance of MultiApp
    CALL app.run()

END

```

Appendix H: Pseudocode for Plagiarism detector.py (Streamlit App)

BEGIN

```

# Import necessary libraries
IMPORT nltk, streamlit, torch, transformers (GPT-J, DeBERTa, T5, BERT)
IMPORT sklearn.metrics (cosine_similarity)
IMPORT string, re, numpy, plotly.graph_objects

```

```

# Download NLTK resources
DOWNLOAD nltk 'punkt'
DOWNLOAD nltk 'stopwords'

# Load AI models and tokenizers
LOAD gptj_tokenizer, gptj_model FROM pretrained 'EleutherAI/gpt-j-6B'
LOAD deberta_tokenizer, deberta_model FROM pretrained
'microsoft/deberta-v2-xlarge' WITH output_hidden_states=True
LOAD t5_tokenizer, t5_model FROM pretrained 't5-small'
LOAD bert_tokenizer, bert_model FROM pretrained 'bert-base-uncased'

# Function to clean input text
FUNCTION clean_text(text):
    REMOVE newlines, extra spaces, and punctuation
    REMOVE special characters
    RETURN cleaned text

# Function to calculate GPT-J perplexity
FUNCTION calculate_perplexity_gptj(text, max_chunk_length=512):
    CLEAN text USING clean_text()
    TOKENIZE text USING gptj_tokenizer
    SPLIT tokens into chunks of max_chunk_length
    FOR each chunk:
        COMPUTE model loss
    SUM total loss and count tokens
    COMPUTE avg loss and perplexity
    RETURN perplexity

# Function to calculate burstiness (word repetition)
FUNCTION calculate_burstiness(text):
    CLEAN text USING clean_text()
    TOKENIZE words
    COMPUTE word frequency
    CALCULATE burstiness score as (repeated words / unique words)
    RETURN burstiness score

# Function to calculate DeBERTa entropy
FUNCTION calculate_entropy_deberta(text):
    CLEAN text USING clean_text()
    TOKENIZE text USING deberta_tokenizer
    COMPUTE softmax probabilities and log probabilities
    CALCULATE entropy as -SUM(probs * log_probs)
    RETURN entropy

# Function to calculate DeBERTa semantic coherence
FUNCTION calculate_semantic_coherence_deberta(text):
    CLEAN text USING clean_text()

```

```

SPLIT text into sentences
COMPUTE sentence embeddings using deberta_model
COMPUTE pairwise cosine similarity between sentence embeddings
RETURN average similarity as coherence score

# Function to calculate T5 perplexity
FUNCTION calculate_perplexity_t5(text):
    CLEAN text USING clean_text()
    TOKENIZE text USING t5_tokenizer
    COMPUTE model loss
    COMPUTE perplexity as exp(loss)
    RETURN perplexity

# Function to calculate BERT entropy
FUNCTION calculate_entropy_bert(text):
    CLEAN text USING clean_text()
    TOKENIZE text USING bert_tokenizer
    COMPUTE softmax probabilities and log probabilities
    CALCULATE entropy as -SUM(probs * log_probs)
    RETURN entropy

# Function to plot probability donut chart
FUNCTION plot_probability_donut_chart(score, is_ai_generated):
    CLAMP score between 0.01 and 0.99
    IF is_ai_generated:
        SET AI probability high (80%-100%)
    ELSE:
        SET Human probability high (80%-100%)
    PLOT probability donut chart using plotly
    DISPLAY chart

# Function to display GPT-J analysis results
FUNCTION display_results_gptj(text):
    COMPUTE perplexity USING calculate_perplexity_gptj()
    COMPUTE burstiness USING calculate_burstiness()
    DETERMINE AI classification IF (perplexity > 10000 AND burstiness >
0.2)
    COMPUTE probability score
    DISPLAY word count, detection scores, and original text
    RETURN perplexity, burstiness score

# Function to display DeBERTa analysis results
FUNCTION display_results_deberta(text):
    COMPUTE entropy USING calculate_entropy_deberta()
    COMPUTE semantic coherence USING
calculate_semantic_coherence_deberta()
    DETERMINE AI classification IF (entropy > 4.5 AND coherence < 0.6)
    COMPUTE probability score

```

```

    DISPLAY word count, detection scores, and original text
    RETURN entropy, semantic coherence

# Function to display T5 analysis results
FUNCTION display_results_t5(text):
    COMPUTE perplexity USING calculate_perplexity_t5()
    DETERMINE AI classification IF (perplexity > 150)
    COMPUTE probability score
    DISPLAY word count, detection scores, and original text
    RETURN perplexity

# Function to display BERT analysis results
FUNCTION display_results_bert(text):
    COMPUTE entropy USING calculate_entropy_bert()
    DETERMINE AI classification IF (entropy < 3.8)
    COMPUTE probability score
    DISPLAY word count, detection scores, and original text
    RETURN entropy

END

```

Appendix I: Pseudocode for Chat History.py (Streamlit App)

```

BEGIN

# Import necessary libraries
IMPORT streamlit AS st

# Function to display chat history
FUNCTION app():
    DISPLAY "Chat History" as title

# Check if chat history exists in session state
IF 'chat_history' EXISTS in session_state AND NOT EMPTY:
    FOR each entry IN session_state['chat_history'], INDEXED FROM 1:
        DISPLAY chat number
        DISPLAY text of the entry
        DISPLAY horizontal line separator

    DISPLAY "Model Used:", entry['model']

# Display relevant metrics based on the model used
IF entry['model'] == "GPT-J":
    IF 'perplexity' EXISTS:
        DISPLAY "Perplexity:", entry['perplexity']
    IF 'burstiness_score' EXISTS:

```



```

        DISPLAY "Burstiness Score:", entry['burstiness_score']

    ELSE IF entry['model'] == "DeBERTa":
        IF 'entropy' EXISTS:
            DISPLAY "Entropy:", entry['entropy']
        IF 'semantic_coherence' EXISTS:
            DISPLAY "Semantic Coherence:", entry['semantic_coherence']

    ELSE IF entry['model'] == "T5":
        IF 'perplexity' EXISTS:
            DISPLAY "Perplexity:", entry['perplexity']

    ELSE IF entry['model'] == "BERT":
        IF 'entropy' EXISTS:
            DISPLAY "Entropy:", entry['entropy']

    # Display AI classification result
    IF 'result_label' EXISTS:
        IF result_label == 1:
            DISPLAY "Result: AI Generated Content" (Red Highlight)
        ELSE:
            DISPLAY "Result: Human Generated Content" (Green
Highlight)

        DISPLAY horizontal line separator

    ELSE:
        DISPLAY "No history available."
        DISPLAY horizontal line separator

    # Run the app if executed as main
    IF script is executed as main:
        CALL app()

END

```